

Ads Sourcing & Matching — Interview Preparation

Background: Apple Ads Team

- **Team focus:** Ads matching/sourcing for App Store, Apple News, Apple Stocks, Apple Sports, Apple Maps
 - **Key differentiators:** Privacy-centric (no IDFA cross-app tracking), premium brand experience, organic ad integration
 - **Maturity:** Earlier stage than Google/Meta/Amazon ads — leaner teams, need breadth + depth
 - **Constraint:** Not pure revenue maximization — must respect privacy + customer journey
-

Part 1: Phone Screen Questions & Answers

Q1: Design a matching system for "photo editor" query against 50K+ apps

Multi-stage retrieval pipeline:

1. Query Understanding

- Spell correction: "phot editor" → "photo editor"
- Query expansion: photo editor → image editor, picture editor (via synonym dict or embedding similarity)
- Intent classification: navigational (specific app name) vs. exploratory (category)

2. Candidate Retrieval (Recall stage) — 50K → top 500~1000

- **Lexical retrieval:** BM25 over inverted index (app name, description, keywords)
- **Semantic retrieval:** Dual encoder (query encoder + app encoder) → ANN search
- Union both result sets for max recall

3. Ranking — 500 → top 20~50

- Cross-encoder or feature-rich model (GBDT/neural ranker)
- Features: semantic score, lexical score, app quality, advertiser bid, historical CTR/CVR

4. Post-ranking

- Policy compliance, budget pacing, frequency cap, diversity, privacy constraints

How to say it in interview:

"I'd design this as a multi-stage funnel. First, a query understanding layer handles spell correction and query expansion — turning 'photo editor' into related terms like 'image editor.' Then for candidate retrieval, I'd run two parallel paths: a lexical path using BM25 over an inverted index for exact

term matching, and a semantic path using a dual encoder model with ANN search — this catches synonyms and paraphrases that BM25 would miss. I'd union both result sets to maximize recall, giving us roughly 500 to 1000 candidates.

Those candidates then go into a ranking stage — a heavier model, maybe a cross-encoder or a feature-rich neural ranker, that considers semantic scores, ad quality, bid price, and historical engagement. Finally, a post-ranking layer applies business rules: budget pacing, frequency caps, diversity, and privacy constraints. Each stage trades off more compute for higher precision on fewer candidates."

Q2: Lexical matching evaluation metrics

Retrieval quality metrics:

- **Recall@K** = $|\text{relevant items in top-K}| / |\text{total relevant items}|$ — coverage
- **Precision@K** = $|\text{relevant items in top-K}| / K$ — accuracy
- **MRR** (Mean Reciprocal Rank) = $1/\text{rank of first relevant result}$
- **NDCG@K** — weighted relevance considering position

The **scoring function itself (BM25)** is what determines "how good is the lexical match" for each query-doc pair.

How to say it in interview:

"For evaluating lexical retrieval, I'd focus on Recall at K — because at the retrieval stage, the priority is coverage: we can't afford to miss relevant ads. Precision at K matters more downstream in ranking. I'd also look at MRR, which measures how quickly we surface the first relevant result, and NDCG if we have graded relevance labels. The retrieval scoring itself comes from BM25, which gives a relevance score for each query-document pair based on term frequency and document frequency statistics."

Q3: BM25 vs TF-IDF — Deep Dive with Examples

TF-IDF — Formula and Term-by-Term Explanation

$$\text{score}(q, d) = \sum_{t \in q} \text{tf}(t, d) \times \text{idf}(t)$$

$\text{tf}(t, d)$ = term frequency — how many times term t appears in document d

- Intuition: more occurrences → more relevant to that term

$\text{idf}(t)$ = inverse document frequency:

$$\text{idf}(t) = \log \frac{N}{\text{df}(t)}$$

- N = total documents in corpus
- $\text{df}(t)$ = number of documents containing term t
- Intuition: rare words have more discriminative power; "the" appears everywhere (low idf), "photoshop" appears in few docs (high idf)

TF-IDF Worked Example

Query = "photo editor", Corpus = 5 apps:

App	Description	Length
A	"photo editor with filters and photo enhancement"	7 words
B	"video editor for social media"	5 words
C	"photo gallery viewer"	3 words
D	"code editor for developers"	4 words
E	"photo photo photo editor editor"	5 words (keyword spam)

Corpus stats:

- $N = 5$
- $\text{df}(\text{"photo"}) = 3$ (A, C, E) $\rightarrow \text{idf} = \log(5/3) = 0.51$
- $\text{df}(\text{"editor"}) = 4$ (A, B, D, E) $\rightarrow \text{idf} = \log(5/4) = 0.22$

Scores:

App A: $\text{tf}(\text{"photo"}, A) = 2$, $\text{tf}(\text{"editor"}, A) = 1$

$\text{score}(q, A) = 2 \times 0.51 + 1 \times 0.22 = 1.24$

App E: $\text{tf}(\text{"photo"}, E) = 3$, $\text{tf}(\text{"editor"}, E) = 2$

$\text{score}(q, E) = 3 \times 0.51 + 2 \times 0.22 = 1.97 \rightarrow \text{HIGHEST! But it's keyword spam!}$

App C: $\text{tf}(\text{"photo"}, C) = 1$, $\text{tf}(\text{"editor"}, C) = 0$

$\text{score}(q, C) = 1 \times 0.51 + 0 \times 0.22 = 0.51$

Problem exposed: App E just stuffs keywords and gets the highest score because TF grows linearly without bound. Also, longer docs naturally accumulate higher TF.

BM25 — Formula and Term-by-Term Explanation

$$\text{score}(q, d) = \sum_{t \in q} \text{idf}(t) \cdot \frac{\text{tf}(t, d) \cdot (k_1 + 1)}{\text{tf}(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgl}}\right)}$$

Breaking down the fraction piece by piece:

Numerator: $\text{tf}(t, d) \cdot (k_1 + 1)$

- Linear scaling of tf, multiplied by a constant $(k_1 + 1)$

Denominator: $\text{tf}(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgl}}\right)$

- This is what creates saturation and length normalization
- As $\text{tf} \rightarrow \infty$: $\frac{\text{tf}}{\text{tf} + \text{constant}} \rightarrow 1.0$ (saturation!)
- The $k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgl}}\right)$ part is the

"effective k_1 " adjusted for doc length

Parameter $k_1 \approx 1.2$ — Saturation control:

- Controls how fast tf saturates
- When $\text{tf} \gg k_1$: the fraction $\frac{k_1}{k_1 + \text{tf}}$, regardless of how much higher tf goes
- When $\text{tf} = 1$: significant contribution
- When $\text{tf} = 10$ vs $\text{tf} = 100$: almost no difference (both saturated)
- $k_1 \rightarrow \infty$: degrades to linear TF-IDF (no saturation)
- $k_1 \rightarrow 0$: becomes binary (term present or not)

Parameter $b \approx 0.75$ — Length normalization:

- $b = 0$: no length normalization at all
- $b = 1$: full normalization proportional to doc length
- $\frac{|d|}{\text{avgdl}}$: if doc is longer than average, denominator increases $\rightarrow$ score decreases
- Intuition: one occurrence of "photo" in a 3-word title is more meaningful than in a 100-word description

avgdl — Average document length across corpus:

- The baseline for "normal" document length
- Docs shorter than avgdl get a boost, docs longer get penalized

BM25 Worked Example

Same corpus. $k_1 = 1.2$, $b = 0.75$, $\text{avgdl} = (7 + 5 + 3 + 4 + 5)/5 = 4.8$

App A ($|d| = 7$): $\text{tf}(\text{"photo"}) = 2$, $\text{tf}(\text{"editor"}) = 1$

Length factor: $1 - b + b \cdot \frac{|d|}{\text{avgdl}} = 1 - 0.75 + 0.75 \cdot \frac{7}{4.8} = 0.25 + 1.09 = 1.34$

$\text{BM25}(\text{"photo"}, A) = 0.51 \cdot \frac{2 \cdot 2.2}{2 + 1.2 \cdot 1.34} = 0.51 \cdot \frac{4.4}{3.61} = 0.51 \cdot 1.22 = 0.62$

$\text{BM25}(\text{"editor"}, A) = 0.22 \cdot \frac{1 \cdot 2.2}{1 + 1.2 \cdot 1.34} = 0.22 \cdot \frac{2.2}{2.61} = 0.22 \cdot 0.84 = 0.19$

$\text{Total}_A = 0.62 + 0.19 = 0.81$

App E ($|d| = 5$): $\text{tf}(\text{"photo"}) = 3$, $\text{tf}(\text{"editor"}) = 2$

Length factor: $1 - 0.75 + 0.75 \cdot \frac{5}{4.8} = 0.25 + 0.78 = 1.03$

$\text{BM25}(\text{"photo"}, E) = 0.51 \cdot \frac{3 \cdot 2.2}{3 + 1.2 \cdot 1.03} = 0.51 \cdot \frac{6.6}{4.24} = 0.51 \cdot 1.56 = 0.79$

$\text{BM25}(\text{"editor"}, E) = 0.22 \cdot \frac{2 \cdot 2.2}{2 + 1.2 \cdot 1.03} = 0.22 \cdot \frac{4.4}{3.24} = 0.22 \cdot 1.36 = 0.30$

$\text{Total}_E = 0.79 + 0.30 = 1.09$

App C ($|d| = 3$): $\text{tf}(\text{"photo"}) = 1$, $\text{tf}(\text{"editor"}) = 0$

Length factor: $1 - 0.75 + 0.75 \times \frac{3}{4.8} = 0.25 + 0.47 = 0.72$

$\text{BM25}(\text{"photo"}, C) = 0.51 \times \frac{1 \times 2.2}{1 + 1.2 \times 0.72} = 0.51 \times \frac{2.2}{1.86} = 0.51 \times 1.18 = 0.60$

$\text{Total}_C = 0.60 + 0 = 0.60$

Comparison TF-IDF vs BM25:

App	TF-IDF	BM25	Note
A	1.24	0.81	Legitimate app
E	1.97	1.09	Keyword spam — gap narrowed significantly!
C	0.51	0.60	Short doc — BM25 gives it relative boost

Key insight: BM25 dramatically reduced the advantage of keyword stuffing (E vs A gap: 59% in TF-IDF → 35% in BM25).

Why BM25's new terms are useful — Summary

BM25 Innovation	What it fixes	Without it
tf saturation (k1)	Keyword spam gets unfair advantage	tf = 100 scores 10x of tf = 10
Length norm (b, avgdl)	Long docs dominate just by being long	A 1000-word description always beats a 5-word title
Probabilistic idf	Better idf formula (handles edge cases)	$\log(N/df)$ can be negative when $df > N/2$

Serving: How BM25 Actually Runs (NOT Embedding-Based!)

BM25 is NOT an embedding method. It does NOT convert query/doc to dense vectors. It uses an **inverted index** — fundamentally different from ANN search.

What is an Inverted Index?

"**Inverted**" means: instead of "doc → which terms it contains" (forward index), we store "term → which docs contain it" (inverted).

Critical: We do NOT store zeros! Only terms that actually appear in a doc are recorded.

Wrong mental model (dense matrix — stores everything including zeros):

	App_Lightroom	App_VSCO	App_Hinge	App_Candy	...
"photo"	2	1	0	0	
"editor"	1	1	0	0	
"dating"	0	0	3	0	
"hinge"	0	0	2	0	

This would be $50,000 \text{ apps} \times 100,000 \text{ terms} = 5 \text{ billion cells}$, 99.9% zeros. Wasteful.

Correct: Inverted Index (only stores non-zero entries):

```
"photo"    → [(Lightroom, tf=2), (VSCO, tf=1), (Snapseed, tf=1)]
"editor"    → [(Lightroom, tf=1), (VSCO, tf=1), (Snapseed, tf=1)]
"dating"    → [(Hinge, tf=1)]
"hinge"     → [(Hinge, tf=1)]
"game"      → [(Candy_Crush, tf=1)]
```

"Hinge" app does NOT appear in "photo"s posting list at all. Not stored as 0 — simply absent. Like a book index: you list pages where a topic appears, not pages where it doesn't.

Offline — Build Inverted Index (Complete Example):

Corpus:

```
App_Lightroom: "professional photo editor and photo enhancement tool"
App_VSCO:      "photo filters and video editor"
App_Hinge:     "hinge dating app designed to be deleted"
App_Candy:     "candy crush saga puzzle game"
App_Snapseed:  "photo editor by google"
```

Step 1: Tokenize each document (remove stopwords like "and", "to", "be"):

```
Lightroom → ["professional", "photo", "editor", "photo", "enhancement",
VSCO      → ["photo", "filters", "video", "editor"]
Hinge     → ["hinge", "dating", "app", "designed", "deleted"]
Candy     → ["candy", "crush", "saga", "puzzle", "game"]
Snapseed  → ["photo", "editor", "google"]
```

Step 2: Process each doc, append to posting lists:

Processing Lightroom:

```
"professional" → [(Lightroom, tf=1)]    ← new entry
"photo"        → [(Lightroom, tf=2)]    ← new entry (appears twice!)
"editor"       → [(Lightroom, tf=1)]    ← new entry
"enhancement"  → [(Lightroom, tf=1)]    ← new entry
"tool"        → [(Lightroom, tf=1)]    ← new entry
```

Processing VSCO (append to existing lists):

```
"photo"    → [(Lightroom, 2), (VSCO, 1)]    ← appended
"filters"   → [(VSCO, 1)]                    ← new entry
"video"     → [(VSCO, 1)]                    ← new entry
"editor"    → [(Lightroom, 1), (VSCO, 1)]    ← appended
```

Processing Hinge:

```
"hinge"     → [(Hinge, 1)]                  ← new entry (completely separate from pho
"dating"    → [(Hinge, 1)]                  ← new entry
"app"       → [(Hinge, 1)]                  ← new entry
"designed"  → [(Hinge, 1)]                  ← new entry
"deleted"   → [(Hinge, 1)]                  ← new entry
```

Note: Hinge's tokens never touch the "photo" or "editor" posting lists. They are completely independent.

Step 3: Final inverted index after all docs processed:

Term	Posting List
"photo"	[(Lightroom,2), (VSCO,1), (Snapseed,1)]
"editor"	[(Lightroom,1), (VSCO,1), (Snapseed,1)]
"professional"	[(Lightroom,1)]
"enhancement"	[(Lightroom,1)]
"tool"	[(Lightroom,1)]
"filters"	[(VSCO,1)]
"video"	[(VSCO,1)]
"hinge"	[(Hinge,1)]
"dating"	[(Hinge,1)]
"app"	[(Hinge,1)]
"designed"	[(Hinge,1)]
"deleted"	[(Hinge,1)]
"candy"	[(Candy,1)]
"crush"	[(Candy,1)]
"game"	[(Candy,1)]
"google"	[(Snapseed,1)]
...	...

Step 4: Pre-compute global statistics:

$N = 5$ (total docs)

$avgdl = (6 + 4 + 5 + 5 + 3) / 5 = 4.6$

Document lengths (after stopword removal):

$|Lightroom| = 6, |VSCO| = 4, |Hinge| = 5, |Candy| = 5, |Snapseed| = 3$

df (= posting list length, instantly available):

$df("photo") = 3$

$df("editor") = 3$

$df("hinge") = 1$

idf (pre-computed from df):

$idf("photo") = \log(5/3) = 0.51$

$idf("editor") = \log(5/3) = 0.51$

$idf("hinge") = \log(5/1) = 1.61$

Yes, idf is pre-computed. Because N and $df(t)$ are fixed after indexing. **Yes, tf is pre-stored** in the posting list entries. Query time just reads it.

Online — Query Time Execution:

Query: "photo editor"

Step 1: Tokenize query → ["photo", "editor"]

Step 2: Look up posting lists (hash table lookup, $O(1)$ per term)

"photo" → [(Lightroom,2), (VSCO,1), (Snapseed,1)]

"editor" → [(Lightroom,1), (VSCO,1), (Snapseed,1)]

Step 3: Candidate set = union of posting lists

candidates = {Lightroom, VSCO, Snapseed}

★ Hinge and Candy are NOT candidates!

★ They don't appear in "photo" or "editor" posting lists.

★ We never even look at them. This is why it's fast!

Step 4: For each candidate, compute BM25 using pre-stored tf, idf, $|d|$

score(Lightroom) = BM25("photo",Lightroom) + BM25("editor",Lightroom)

score(VSCO) = BM25("photo",VSCO) + BM25("editor",VSCO)

score(Snapseed) = BM25("photo",Snapseed) + BM25("editor",Snapseed)

(All values – tf, idf, $|d|$, avgdl – are pre-stored. Just plug into fo

Step 5: Sort by score → return top-K

Query: "hinge dating"

Step 1: Tokenize → ["hinge", "dating"]

Step 2: Look up posting lists

"hinge" → [(Hinge, tf=1)]

"dating" → [(Hinge, tf=1)]

Step 3: candidates = {Hinge} ← only 1 candidate!

Lightroom, VSCO, Snapseed, Candy → not considered at all.

Step 4: score(Hinge) = BM25("hinge",Hinge) + BM25("dating",Hinge)

Step 5: Return [Hinge]

Why Inverted Index is fast — the key insight:

Method	What it computes	# computations for "photo editor"
Brute force	Score ALL 50,000 apps	50,000
Inverted index	Only score apps in posting lists	~3,000 (apps containing "photo" or "editor")

For rare queries like "hinge dating": posting lists may only have 50 entries → 50 computations instead of 50,000. **1000x speedup.**

The fundamental trick: **if a query term doesn't appear in a doc, that doc's BM25 contribution for that term = 0. So skip it entirely.**

Why NOT convert to TF-IDF embedding + cosine?

Some people confuse "TF-IDF vector" with dense embedding:

- A TF-IDF vector is $|V|$ -dimensional (V = vocab size, e.g. 100,000 dims)
- It's extremely **sparse** (most dimensions are 0)
- You COULD compute cosine similarity between query TF-IDF vector and all doc TF-IDF vectors
- But this is **slower** than inverted index lookup! Inverted index skips all terms not in the query
- Inverted index: $O(|\text{query terms}| \times \text{avg posting list length})$
- Dense vector cosine: $O(N \times V)$ — much worse

Bottom line: BM25 serving = inverted index lookup + scoring formula. Fast, simple, no ML model needed, no training needed. That's why it's the universal baseline.

FAQ — BM25 Serving

Q: Do we store (hinge, 0) in "photo"s posting list?

No! Only non-zero entries are stored. Hinge has no record under "photo."

Q: Is idf pre-computed?

Yes. $\text{idf}(t) = \log(N / \text{df}(t))$. Both N and df are fixed after indexing.

Q: Is tf pre-computed?

Yes. Stored in the posting list: $(\text{doc_id}, \text{tf})$. Query time just reads it.

Q: What if a new app is added?

Tokenize it, append to relevant posting lists, update N , avgdl , and affected df values. Incremental — no full rebuild needed.

Q: What if query contains a term that never appeared in any app (OOV)?

BM25 returns **zero results** for that term. If ALL query terms are OOV, the candidate set is empty.

Example: query = "hahaha"

- "hahaha" is not in the inverted index (no app ever contained this token)
- Posting list lookup returns empty
- Candidates = $\{\}$ → no results

This is the fundamental limitation of lexical matching: **if a token was never seen in the corpus, BM25 cannot do anything with it.** This is a primary motivation for why semantic retrieval is needed alongside BM25.

Intuitive Summary of the Entire BM25 Serving Flow

1. **Offline:** for each app, tokenize and count each token's frequency (tf) within that app. Build a reverse mapping: token → list of (app, tf). Also pre-compute each token's idf

from how many apps contain it.

2. **Online:** query comes in → tokenize → for each query token, look up the reverse mapping to find which apps contain it and their pre-stored tf. The candidate set is only apps that share at least one token with the query. Compute BM25 score using the pre-stored tf, idf, doc length. Apps that share zero tokens with the query are never even considered — this is why it's fast.
3. **Edge case:** if a query token never appeared in any app's corpus, the posting list is empty. BM25 has zero candidates for that token. If the entire query is OOV, BM25 returns nothing.

Why BM25 Alone is Not Enough — Motivation for Semantic Retrieval

BM25's OOV problem is just ONE reason semantic retrieval is needed. Here are ALL the reasons:

BM25 Limitation	Example	Semantic retrieval handles it because
OOV (out-of-vocabulary)	Query "hahaha" → 0 results	Encoder maps any text to an embedding; even unseen tokens get a vector
Synonyms	"picture" vs "photo" — different tokens, BM25 can't match	Encoder learns "picture" ≈ "photo" in embedding space
Paraphrases	"app to edit my photos" vs app titled "Lightroom"	Encoder understands intent despite zero token overlap
Abbreviations/slang	"pic editor" or "fotoshop"	Trained on noisy text, learns common variations
Conceptual matching	"make my selfie look better" → photo editing apps	Encoder captures intent-level semantics
Multilingual	Query in Spanish, app name in English	Multilingual encoders map both to same space
Misspellings	"camra" — not in index	Subword tokenization (WordPiece) partially handles; embedding is close to "camera"

The deeper reason: BM25 matches on **surface form** (exact tokens). Semantic retrieval matches on **meaning** (learned representations). These are fundamentally complementary:

- BM25 is best when: user types the exact app name ("Instagram"), exact keyword match matters
- Semantic is best when: user describes what they want in natural language ("app to meet new people")

Neither alone is sufficient. BM25 has perfect precision for exact matches but zero recall for semantic gaps. Semantic has broad recall but may miss exact token matches that BM25 nails. This is why production systems always run both in parallel and union the results.

Do BM25/TF-IDF need training?

NO. They are completely unsupervised:

- Only need corpus statistics (document lengths, term frequencies, doc frequencies)
- k1 and b have good defaults (1.2, 0.75); can optionally tune on validation set
- No gradient descent, no GPU, no labeled data
- This is why BM25 remains the #1 first-stage retrieval baseline even in 2026

How to say it in interview (BM25 vs TF-IDF):

"Both TF-IDF and BM25 are term-based scoring functions that don't need any training — they only rely on corpus statistics. The key difference is that TF-IDF has a linear term frequency component, which means keyword stuffing gets rewarded: if an app repeats 'photo' 100 times, it scores much higher than one that mentions it twice. BM25 fixes this with two innovations: first, a saturation function controlled by parameter k1 that gives diminishing returns after a term appears enough times — so repeating 'photo' 100 times barely helps more than 10 times. Second, a length normalization controlled by parameter b that penalizes long documents proportionally — one mention of 'photo' in a 3-word title is more informative than in a 1000-word description.

For serving, BM25 uses an inverted index, not embeddings. Offline, we tokenize each app and build a reverse mapping: term → list of apps that contain it with their tf. Online, when a query comes in, we tokenize it, look up each query token in the inverted index, and only score apps that actually contain at least one query token. This means we skip all irrelevant apps entirely — if a query is 'photo editor,' we never even look at a dating app because it doesn't appear in those posting lists. That's what makes it fast. And it needs zero training — no gradients, no GPU, just corpus statistics."

How to say it (Why BM25 isn't enough, need semantic):

"BM25 fundamentally matches on surface form — exact token overlap. So if a user searches 'picture editor' but an app is titled 'Lightroom,' BM25 gives zero score because there's no token overlap. Similarly, paraphrases like 'make my photos look better' won't match any specific app name. That's why we need semantic retrieval in parallel — a dual encoder that maps both query and app descriptions into the same embedding space, so semantically similar things end up close together regardless of exact wording. The two approaches are complementary: BM25 nails exact matches with perfect precision, semantic retrieval catches everything BM25 misses."

Q4: Why Dual Encoder over Cross-Encoder?

Architecture Comparison

	Dual Encoder	Cross-Encoder
Architecture	Query and Doc encoded separately into embeddings, score = dot product / cosine	Query + Doc concatenated, fed into single transformer, output = relevance score
Interaction	No cross-attention between query and doc	Full cross-attention captures fine-grained token interactions
Offline compute	Doc embeddings pre-computed and stored	Must re-compute for every (query, doc) pair

Online latency	Encode query (~5ms) + ANN lookup (~5ms) = ~10ms total	Each pair requires full model forward pass (~20-50ms × N candidates)
Scalability	Millions of candidates via ANN	Only feasible for top 50-500 candidates
Quality	Good but limited by independence assumption	Higher quality due to cross-attention

Why Dual Encoder for Retrieval:

1. **Latency** — Cannot run cross-encoder on 50K candidates; dual encoder + ANN handles it in <10ms
2. **Decoupling** — App embeddings computed offline, stored in vector index (FAISS/ScaNN); only query encoding is real-time
3. **Incremental updates** — New app comes in → compute one embedding → add to index; no full reindex needed
4. **Multi-use** — Same app embeddings can serve different retrieval needs

Not *only* latency — other reasons:

- **Caching** — App embeddings don't change with each query, massive compute savings
- **Independence** — Can update query encoder and app encoder on different schedules
- **Infrastructure simplicity** — Standard ANN serving infra (FAISS, ScaNN, Milvus, Pinecone)

Practical architecture:

Retrieval: Dual Encoder + ANN → top 500 candidates (recall-focused)
 Ranking: Cross-Encoder → top 20 candidates (precision-focused)

How to say it in interview:

"I'd use a dual encoder for retrieval rather than a cross-encoder, primarily because of scalability. A cross-encoder concatenates the query and document together and passes them through a single transformer — this gives the highest quality because every query token can attend to every document token. But it means you have to run the full model for every query-document pair. With 50,000 candidates, that's 50,000 forward passes per query — way too slow for real-time serving.

A dual encoder solves this by encoding query and documents independently into separate embeddings. The key insight is: document embeddings don't depend on the query, so we pre-compute and cache all 50K app embeddings offline. At serving time, we only need to encode the query — one forward pass — then do a dot product search via ANN, which takes under 10 milliseconds. So it's not just latency — it's also about decoupling: we can update the app index incrementally when new apps arrive, without re-running the query encoder.

The trade-off is quality. Since query and document never 'see' each other's tokens during encoding, the model can't capture fine-grained interactions. That's why we use the cross-encoder downstream in ranking — on just the top

100-500 candidates where we can afford the per-pair inference cost. So it's dual encoder for recall at the top of the funnel, cross-encoder for precision at the bottom."

Q5: Recall, Precision, ROC-AUC, PR-AUC

Confusion Matrix Basics

For any binary prediction (relevant/irrelevant):

	Predicted Positive	Predicted Negative
Actually Positive	TP (True Positive)	FN (False Negative)
Actually Negative	FP (False Positive)	TN (True Negative)

Precision and Recall — Two formulations

Using TP/FP/FN:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- "Of everything I predicted positive, how many are actually positive?"
- Denominator = all items you predicted as positive (returned/retrieved)

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- "Of everything that IS positive, how many did I find?"
- Denominator = all actually positive items (whether you found them or not)

Using retrieval notation (equivalent, applied to top-K):

$$\text{Precision@K} = \frac{|\{\text{relevant}\} \cap \{\text{top-K returned}\}|}{|\{\text{top-K returned}\}|}$$

$$\text{Recall@K} = \frac{|\{\text{relevant}\} \cap \{\text{top-K returned}\}|}{|\{\text{all relevant in corpus}\}|}$$

Mapping between the two:

- \$TP\$ = relevant items in your top-K
- \$FP\$ = irrelevant items in your top-K
- \$FN\$ = relevant items you missed (not in top-K)
- \$TN\$ = irrelevant items you correctly didn't return

In ads matching context:

- **Retrieval stage** optimizes for **Recall** — must not miss relevant ads (cast a wide net). A few irrelevant candidates in the pool is OK, ranking will filter them.
- **Ranking stage** optimizes for **Precision** / **NDCG** — what's actually shown to users must be high quality.

ROC-AUC vs PR-AUC — Which is useful for sourcing/ranking?

ROC Curve (Receiver Operating Characteristic)

ROC plots: $\text{TPR (Recall)} = \frac{\text{TP}}{\text{TP} + \text{FN}}$ vs $\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$

- X-axis: False Positive Rate (FPR)
- Y-axis: True Positive Rate (TPR = Recall)
- AUC = area under this curve (0.5 = random, 1.0 = perfect)

Problem with ROC-AUC for ads/retrieval:

ROC-AUC uses **TN (True Negatives)** in FPR's denominator. In retrieval:

- Corpus = 50,000 apps
- Relevant to "photo editor" = maybe 200 apps
- TN = 49,800 irrelevant apps you correctly didn't return

TN is **massive** (49,800). This makes FPR almost always near 0, regardless of model quality. ROC-AUC stays high even for bad models because the denominator ($\text{FP} + \text{TN}$) is dominated by the huge TN count.

Example: Model retrieves 500 apps, 100 are relevant, 400 are irrelevant:

- $\text{FPR} = \frac{400}{400 + 49,400} = 0.008$ — looks great! But you returned 400 irrelevant items.

PR Curve (Precision-Recall Curve)

PR plots: $\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$ vs $\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$

- X-axis: Recall
- Y-axis: Precision
- AUC = area under this curve (higher = better)

Why PR-AUC is better for ads/retrieval/ranking:

PR-AUC does **NOT use TN at all!** Neither Precision nor Recall involves TN:

- $\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$ — no TN
- $\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$ — no TN

This means PR-AUC is **not fooled by the massive number of irrelevant items** in the corpus. It directly measures: "among what you returned (TP + FP), how good is it?" and "of what exists (TP + FN), how much did you find?"

Summary: When to use which?

Metric	Uses TN?	Good for	Bad for
ROC-AUC	Yes (in FPR denominator)	Balanced datasets (50/50 pos/neg)	Imbalanced data (retrieval, ads, fraud)
PR-AUC	No	Imbalanced datasets where positives are rare	Balanced datasets (both work, ROC is simpler)

For ads sourcing/retrieval:

- Relevant apps = ~ 200 out of 50,000 $\rightarrow$ **extremely imbalanced** (0.4% positive rate)
- ROC-AUC will be misleadingly high (> 0.95) even for mediocre models
- **PR-AUC is the correct metric** — it directly penalizes irrelevant retrievals

For ranking (after retrieval):

- Candidate set is already filtered to ~ 500 , maybe 100 are relevant $\rightarrow$ still imbalanced (20%)
- **PR-AUC still more informative** than ROC-AUC
- But in practice, ranking uses **NDCG** (considers position) which is even better

Practical usage in ads systems:

Stage	Primary metrics	Why
Retrieval model (offline eval)	Recall@K, PR-AUC	Must find all relevant items; PR-AUC for overall quality
Relevance model (offline eval)	Precision@K, PR-AUC, F1	Must accurately judge relevance
Ranking model (offline eval)	NDCG@K, MAP	Position-aware metrics for ranked lists
All models (online eval)	CTR, CVR, Revenue	Business outcomes via A/B test

Why PR-AUC > ROC-AUC for retrieval — Intuitive explanation

Imagine two retrieval models:

- **Model A:** retrieves 500 apps, 100 relevant + 400 irrelevant
- **Model B:** retrieves 500 apps, 150 relevant + 350 irrelevant

ROC-AUC perspective:

- Both have $\text{FPR} \approx 0.008$ (because $\text{TN} \approx 49,500$ dominates). ROC barely distinguishes them.

PR-AUC perspective:

- Model A: Precision = $100/500 = 0.20$
- Model B: Precision = $150/500 = 0.30$ — clearly better, and PR curve shows it.

The core insight: In retrieval, you don't care about TN (apps you correctly ignored). You care about TP vs FP (quality of what you retrieved) and TP vs FN (coverage of relevant items). PR-AUC measures exactly these two things.

How to say it in interview (Precision/Recall):

"Precision asks: of everything I returned, how much is actually relevant? The denominator is TP plus FP — the total number of items I retrieved. Recall asks: of everything that IS relevant in the entire corpus, how much did I find? The denominator is TP plus FN — all relevant items whether I found them or

not. For retrieval, we care most about recall because we can't rank what we never retrieved. For ranking, precision and NDCG matter more because what we show the user must be high quality."

How to say it (PR-AUC vs ROC-AUC):

"I'd use PR-AUC over ROC-AUC for evaluating retrieval models. The reason is that ROC-AUC uses true negatives in the false positive rate denominator. In retrieval, we have 50,000 apps but maybe only 200 are relevant to any given query — so true negatives dominate at 49,800. This means the false positive rate stays near zero regardless of model quality, making ROC-AUC misleadingly high even for bad models."

PR-AUC avoids this entirely — neither precision nor recall use true negatives in their formulas. Precision is TP over TP plus FP, recall is TP over TP plus FN. So PR-AUC directly measures what we actually care about: the quality of what we retrieved and the coverage of what exists. For any highly imbalanced problem like retrieval, where positive examples are rare, PR-AUC is the more informative metric."

Q6: Revenue up but relevance drops — how to fix?

Root cause: Dual encoder trained on conversion (click/download) data learns "what gets clicked" not "what is relevant." Popular/clickbait apps dominate; niche-but-relevant apps get suppressed.

Solution 1: Multi-objective training

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{conversion}}(\text{conversion pairs}) + \beta \cdot \mathcal{L}_{\text{relevance}}(\text{relevance pairs})$$

- Get relevance labels from human annotation or LLM-as-judge
- Train model to balance both signals

Solution 2: Relevance guardrail model

- Train a separate **relevance classifier** (can be cross-encoder for higher quality)
- After retrieval, filter out candidates with `relevance_score < threshold`
- Acts as a hard gate before ranking

Solution 3: Constrained optimization in ranking

$$\max_{\theta} \mathbb{E}[\text{revenue}] \quad \text{s.t.} \quad \text{relevance_score} \geq \theta; \quad \forall \text{served ads}$$

Or soft version:

$$\text{final_score} = \lambda \cdot \text{revenue_score} + (1 - \lambda) \cdot \text{relevance_score}$$

Solution 4: Training data debiasing

- Add **relevance-annotated hard negatives** — apps that were clicked but human-judged irrelevant
- Reduce weight on noisy click signals
- Use impression-weighted sampling to account for position bias

Solution 5: Separate retrieval paths

- Keep conversion-optimized retrieval path for revenue
- Add relevance-optimized retrieval path (trained on relevance labels)
- Merge results with relevance minimum threshold enforced

Apple-specific consideration:

- Privacy constraints naturally push toward **contextual relevance** (query-app semantic match) over behavioral exploitation
- This aligns business model with relevance — feature not bug

How to say it in interview:

"This is a classic tension in ads systems — our retrieval model is trained on conversion data like clicks and downloads, so it learns 'what gets engaged with' rather than 'what's truly relevant.' Popular or clickbait apps get over-promoted even when they don't match the query well.

I'd address this at multiple levels. First, in training: add a relevance objective to the loss function alongside conversion — make it multi-objective so the model balances both signals. The relevance labels can come from human annotation or LLM-as-judge scoring.

Second, add a standalone relevance model as a quality gate after retrieval. This can be a cross-encoder trained purely on relevance labels — it acts as a hard filter: if an app scores below a relevance threshold, it gets dropped regardless of how high its revenue potential is.

Third, in the ranking stage, use constrained optimization — maximize expected revenue subject to a minimum relevance score. This way we're not giving up revenue entirely, but we're guaranteeing a baseline of quality.

For Apple specifically, the privacy-centric approach actually helps here — since we can't rely heavily on user behavioral data, we're naturally pushed toward contextual relevance based on query-app semantic matching, which tends to correlate with true relevance better than pure behavioral signals."

Part 2: Predicted Follow-up Questions

Q7: How to train a Dual Encoder? Data? Labels? Loss Design?

Training data sources (from search logs):

Signal	Label Type	Strength	Volume
--------	------------	----------	--------

Downloaded app	Strong positive	High quality, clear intent	Sparse
Clicked app	Weak positive	Noisy (curiosity clicks, misclicks)	Abundant
Impressed but not clicked	Weak negative	Very noisy (position bias)	Abundant
Random app from corpus	Easy negative	Clean but uninformative	Unlimited
BM25 top result not clicked	Hard negative	Informative for model	Moderate

Position Bias in Training Data — Why Retrieval Models Are Also Affected

Common misconception: "Position bias only affects ranking models (CTR prediction). Retrieval doesn't have this problem."

Wrong. Any model trained on behavioral labels (click/install) is affected, including retrieval dual encoders.

Why retrieval has position bias:

The training labels for retrieval come from click/install logs. But clicks are **confounded by position**:

Position 1 app → user sees it → clicks it → becomes POSITIVE label
 Position 8 app → user never scrolls down → no click → becomes NEGATIVE

But the position-8 app might be MORE relevant! It just wasn't seen.

So your training data is polluted:

- Some "positives" are just things that happened to be in position 1 (not necessarily relevant)
- Some "negatives" are actually highly relevant but were in low positions (never seen by user)

This directly hurts retrieval model quality: the model learns "things that the previous system ranked high" rather than "things that are truly relevant to the query."

Solutions for retrieval models:

1. Inverse Propensity Weighting (IPW):

$$w_i = \frac{1}{P(\text{click} \mid \text{position}_i)}$$

- Clicks from lower positions get higher weight (user overcame position disadvantage → stronger relevance signal)
- Clicks from position 1 get lower weight (might just be position effect)

How to compute $P(\text{click} \mid \text{position})$:

Simply aggregate from historical logs — count the average CTR at each position across all queries:

$$P(\text{click} \mid \text{pos} = k) = \frac{\text{total clicks at position } k}{\text{total impressions at position } k}$$

Example from logs (aggregated over millions of queries):

	Position	Impressions	Clicks	$P(\text{click} \mid \text{pos})$	IPW weight $w = 1/P$
1		10,000,000	3,000,000	0.30	3.3
2		10,000,000	2,000,000	0.20	5.0
3		9,000,000	1,350,000	0.15	6.7
5		7,000,000	700,000	0.10	10.0
8		4,000,000	200,000	0.05	20.0

Intuition: Position 1 has 30% CTR (high, partly because it's just "seen first"). Position 8 has 5% CTR. A click at position 8 is **much** more meaningful — the user scrolled all the way down and still clicked → strong relevance signal. So it gets 20x weight vs 3.3x for position 1.

How the weight w_i is used in training:

It multiplies the loss for that training sample:

$$\mathcal{L}_{\text{debiased}} = \frac{1}{N} \sum_{i=1}^N w_i \cdot \mathcal{L}_i$$

Worked example:

Training batch has 3 clicked (query, app) pairs:

Sample	Query	App clicked	Position	$P(\text{click} \mid \text{pos})$	w_i
1	"photo editor"	Lightroom	pos = 1	0.30	3.3
2	"photo editor"	VSCO	pos = 3	0.15	6.7
3	"photo editor"	Darkroom	pos = 8	0.05	20.0

Without IPW: all 3 samples contribute equally to the loss. Model treats position-1 click as just as informative as position-8 click.

With IPW:

- Lightroom (pos 1): loss $\times 3.3 \rightarrow$ low weight, maybe user just clicked because it was first
- VSCO (pos 3): loss $\times 6.7 \rightarrow$ medium weight
- Darkroom (pos 8): loss $\times 20.0 \rightarrow$ high weight, user really wanted this app

$$\mathcal{L} = \frac{1}{3}(3.3 \cdot \mathcal{L}_1 + 6.7 \cdot \mathcal{L}_2 + 20.0 \cdot \mathcal{L}_3)$$

The model learns more from Darkroom's click signal (it's a higher-confidence positive) and less from Lightroom's (it might be position effect, not true relevance).

In practice, weights are often normalized or clipped:

- Normalize: $w_i' = w_i / \sum w_j$ (prevents extreme weights from dominating)
- Clip: $w_i' = \min(w_i, w_{\max})$ where $w_{\max} = 20$ (prevents rare-position samples from exploding gradients)

2. Position as feature (for ranking, less common for retrieval):

- Add position as input → model learns to deconfound
- At inference, set position = constant for all candidates

3. Only use strong signals (downloads > clicks):

- Downloads are less position-biased than clicks (user had to actively install)
- Trade-off: much less training data

4. Randomization / Exploration:

- Occasionally show random orderings → collect unbiased click data
- Expensive (hurts short-term revenue) but gives clean labels

5. Use "impression but not clicked" negatives carefully:

- Only treat top-3 positions not clicked as negatives (user definitely saw them)
- Don't use position 8+ as negatives (user might not have scrolled there)

How to say it in interview:

"Position bias isn't just a ranking problem — it affects retrieval too. Our training labels come from clicks and installs, but clicks are confounded by position. An app in position 1 gets clicked because it's seen, not necessarily because it's the most relevant. An app in position 8 might be more relevant but never gets clicked because the user didn't scroll."

So when we train a retrieval model on this data, it learns 'what the previous system ranked high' rather than 'what's truly relevant.' The main fix I'd use is Inverse Propensity Weighting — upweight clicks from lower positions since those users overcame a position disadvantage, indicating stronger intent. I'd also be careful with negative labels: only count an impressed-but-not-clicked app as negative if it was in a top position where the user definitely saw it."

How to handle strong vs weak positives in training?

Approach 1: Weighted InfoNCE (most common in industry)

All positives go into the same contrastive loss, but with different importance weights:

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(q, d^+))}{\sum_j \exp(\text{sim}(q, d_j))}$$

$$\mathcal{L}_{\text{weighted}} = \frac{1}{N} \sum_i w_i \mathcal{L}_i$$

Where weights w_i : download = 2.0 (strong positive), click = 1.0 (weak positive).

Approach 2: Cosine Similarity MSE with soft labels (your suggestion — valid!)

$$\mathcal{L} = \frac{1}{N} \sum_i \left(\cos(\mathbf{q}_i, \mathbf{d}_i) - y_i \right)^2$$

Where soft labels y_i :

- download $\rightarrow y = 1.0$
- click $\rightarrow y = 0.7$
- impression no click $\rightarrow y = 0.2$
- random negative $\rightarrow y = 0.0$

Pros of soft label approach:

- Naturally encodes graded relevance (0.7 vs 1.0 vs 0.2)
- Smoother gradient signal
- No need to discretize into pos/neg

Cons vs InfoNCE:

- InfoNCE does implicit hard negative mining via in-batch contrast
- InfoNCE empirically outperforms MSE/cosine loss in most retrieval benchmarks
- MSE loss doesn't push negatives apart as aggressively

Approach 3: Curriculum / Multi-stage training (practical best)

Stage 1: Pre-train on large click data (abundant, noisy)

→ Model learns basic semantic matching

Stage 2: Fine-tune on download data (sparse, high quality)

→ Model refines toward conversion-relevant matching

Stage 3 (optional): Fine-tune on human relevance labels

→ Model incorporates true relevance signal

Loss Functions Compared

Loss	Formula	Best for
InfoNCE	$-\log \frac{e^{s^+} + \tau}{\sum_j e^{s_j} + \tau}$	Standard contrastive, hard 1/0 labels
Cosine MSE	$ \cos(\mathbf{q}, \mathbf{d}) - y ^2$	Soft/graded labels
Triplet	$\max(0, s^+ - s^- + m)$	When you have explicit (q, pos, neg) triples
Circle Loss	Unified on similarity pairs	Flexible hard/soft distinction

Multi-Objective Dual Encoder — How It Works

Key constraint: In retrieval, you can only have ONE similarity score for ANN search. So multi-objective must be in the **loss**, not in separate output heads.

Same dual encoder, multiple loss terms on the same embedding space:

$$\text{sim}(q, d) = \mathbf{q}^T \mathbf{d} = \text{QueryEncoder}(q)^T \cdot \text{AppEncoder}(d)$$

$$\mathcal{L}_{\text{click}} = \text{InfoNCE}(\text{sim}, \text{positives} = \text{clicked apps}, \text{negatives} = \text{non-clicked})$$

$$\mathcal{L}_{\text{download}} = \text{InfoNCE}(\text{sim}, \text{positives} = \text{downloaded apps}, \text{negatives} = \text{non-downloaded})$$

$$\mathcal{L}_{\text{relevance}} = \text{InfoNCE}(\text{sim}, \text{positives} = \text{human-relevant apps}, \text{negatives} = \text{irrelevant})$$

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{click}} + \beta \cdot \mathcal{L}_{\text{download}} + \gamma \cdot \mathcal{L}_{\text{relevance}}$$

Where α , β , γ are hyperparameters balancing the objectives.

Why this works: The embedding space learns a "blended" notion of similarity that satisfies all three objectives simultaneously. At serving time, one ANN lookup retrieves candidates that are likely to be clicked AND downloaded AND relevant.

Contrast with ranking multi-objective (MMOE/PLE):

- Ranking models CAN have separate output heads (pCTR head, pCVR head, relevance head) because they score each candidate independently — no ANN constraint
- Retrieval dual encoder CANNOT — must produce a single embedding for ANN

Architecture details:

Query Encoder: BERT-base → mean pooling → 768-dim → linear projection →
App Encoder: BERT-base → mean pooling → 768-dim → linear projection →
(input = app_title + " [SEP] " + description[:128])

$$\text{sim}(q, d) = \frac{\mathbf{q}^T \mathbf{d}}{\tau}$$

(dot product preferred over cosine for ANN efficiency in some systems; τ = temperature)

Training pipeline:

1. Collect search logs → (query, impressed apps, clicked apps, downloaded apps)
2. Build training samples: (query, positive_app, [negative_apps])
3. Initialize encoders from pre-trained LM (BERT, sentence-transformers, E5)
4. Fine-tune with multi-objective contrastive loss + hard negative mining
5. Evaluate on held-out relevance judgments (Recall@K, MRR)
6. Periodic re-training (weekly/monthly) as user behavior shifts

How to say it in interview (Dual Encoder Training):

"For training data, we mine positive pairs from search logs — downloads are strong positives, clicks are weak positives. For negatives, I'd use a mix:

random negatives from the corpus for baseline contrast, BM25 top results that weren't clicked as medium-hard negatives, and in-batch negatives which come for free.

The loss function is InfoNCE — essentially a softmax over similarities. You compute the dot product between the query embedding and all candidate embeddings, then push the positive pair's similarity up while pushing negatives down. It's the same as a multi-class classification: 'which of these N items is the true positive?'

For handling different label strengths — clicks versus downloads — I'd use weighted InfoNCE: same loss formula but give download pairs a higher gradient weight, say 2x compared to clicks. You could also use a cosine MSE loss with soft labels like 1.0 for downloads, 0.7 for clicks, 0.2 for impressions without clicks — this lets the model learn graded relevance rather than binary.

For multi-objective training, the key constraint is that a retrieval model can only output a single similarity score for ANN search. So we can't have separate output heads like in a ranking model. Instead, we add multiple loss terms on the same embedding space — a click loss, a download loss, and a relevance loss — weighted and summed. The embedding learns a blended representation that tries to satisfy all three objectives simultaneously."

How to say it (Hard Negatives):

"Hard negatives are samples that are NOT relevant but LOOK relevant to the model — high similarity despite being negative. They're critical because with only easy random negatives, the model quickly gets 99% accuracy and stops learning. The loss goes to zero, gradients vanish.

Hard negatives force the model to learn fine-grained distinctions. For 'photo editor,' a hard negative might be 'Photo Gallery Viewer' — lexically similar but functionally different. The model has to learn that 'editor' and 'gallery' imply different purposes.

I'd source them from three places: BM25 top results that weren't clicked — these are things BM25 thinks are relevant but users didn't engage with. In-batch negatives that happen to score high. And iterative model-based mining: after one round of training, use the model itself to find its own top results that are actually negative, then retrain. This is the ANCE approach — you mine negatives from the model's own confusion.

Importantly, hard negative mining is a sampling strategy, not a labeling strategy. The labels don't change — positive is still clicked or downloaded, negative is still not. We're just choosing WHICH negatives to show the model during training."

Q7b: Standalone Relevance Model — Design and Labels

Why a separate relevance model?

	Multi-objective dual encoder	Standalone relevance model
Architecture	One model, relevance blended into loss	Separate model, runs after retrieval
Model type	Must be dual encoder (ANN constraint)	Can be heavier: cross-encoder, GBDT
Quality	Relevance diluted by revenue objectives	Dedicated, higher quality
Serving	Single ANN lookup	Extra model call on ~500 candidates
Use case	Bake relevance into retrieval	Hard filter/feature for ranking

How to get relevance labels (the hard problem):

Method 1: Human annotation (gold standard)

Annotator task:

Given: query = "photo editor", app = "Instagram"

Question: How relevant is this app to this query?

Scale: 0-4

4 = Perfect match (e.g., Adobe Lightroom)

3 = Excellent (e.g., VSCO - primarily a photo editor)

2 = Good (e.g., Instagram - has editing but not primary purpose)

1 = Fair (e.g., Camera app - related but not editor)

0 = Bad (e.g., Candy Crush - irrelevant)

Volume needed: 50K-200K labeled pairs for a good model

Cost: \$0.10-0.50 per judgment

Method 2: LLM-as-judge (scalable, 90%+ agreement with humans)

prompt = f"""Rate relevance of this app to the search query.

Query: "{query}"

App: "{app_name}" - {app_description}

Scale 0-4 (4=perfect match, 0=irrelevant). Output only the number."""

Run GPT-4/Claude on 1M+ pairs at ~\$0.01 per judgment

Method 3: Behavioral heuristics (noisy but free)

Relevance proxy signals:

- Query is exact/partial match of app name → high relevance
- App category matches query intent → medium
- Downloaded and retained 7+ days → strong relevance
- Downloaded and uninstalled within 24h → weak relevance
- High dwell time on app page after click → relevant
- Quick bounce (< 3s on page) → irrelevant

Method 4: Distillation from heavy model

- Train a cross-encoder on small human-labeled dataset (50K pairs)
- Use cross-encoder to score 5M+ pairs (offline, slow but accurate)
- Train lightweight dual encoder / classifier on these "teacher" scores

Relevance model architecture options:

Option A: Cross-encoder (highest quality, used as filter after retrieval)
Input: "[CLS] query [SEP] app_title app_description [SEP]"
Output: `relevance_score` $\in [0, 1]$
Latency: $\sim 5\text{ms}$ per pair $\times 500$ candidates = 2.5s $\rightarrow$ too slow for all, us

Option B: Lightweight classifier (faster, use as feature in ranker)
Input: pre-computed features (BM25 score, category match, embedding s
Model: 2-layer MLP or small GBDT
Output: `relevance_score`
Latency: $\sim 0.1\text{ms}$ per candidate $\rightarrow$ feasible for 500 candidates

How to say it in interview (Standalone Relevance Model):

"I'd build a standalone relevance model as a quality gate after retrieval. The reason is: the retrieval model is multi-objective — it optimizes for clicks, downloads, AND relevance in one embedding space. That's inherently a compromise. A dedicated relevance model gives 100% capacity to one question: is this app actually relevant to this query?"

For labels, the gold standard is human annotation — annotators rate query-app pairs on a 0-4 relevance scale. But at scale, I'd use LLM-as-judge to generate millions of labels at a fraction of the cost, with about 90% agreement with humans. The model itself can be a cross-encoder since it runs on only 100-500 candidates after retrieval — we can afford the per-pair inference cost at that scale. If latency is still too tight, I'd distill the cross-encoder into a lightweight MLP or GBDT."

Q7c: Hard Negative Mining — Detailed Explanation

What is a hard negative?

Hard negative = a sample that is NOT relevant to the query but LOOKS relevant to the model (high similarity score despite being negative).

Easy negative: ("photo editor", "Candy Crush") $\rightarrow$ trivially different

Hard negative: ("photo editor", "Photo Gallery") $\rightarrow$ lexically similar

Why hard negatives matter:

With only random/easy negatives:

- Model quickly achieves 99% accuracy (too easy)
- Loss $\rightarrow 0$, gradients $\rightarrow 0$, no more learning
- Model cannot distinguish between "photo editor" and "photo viewer" — both score high

With hard negatives:

- Model is forced to learn fine-grained distinctions
- "photo" alone is not enough — must understand "editor" functionality vs "viewer" vs "gallery"
- Much better retrieval quality in practice

How to find hard negatives — 4 methods:

Method 1: BM25 top results not clicked (most common, free)

Query: "photo editor"

BM25 retrieves: [App_A(clicked), App_B(not clicked), App_C(not clicked)]

Hard negatives for this query = {App_B, App_C}

Label logic: BM25 thinks they're relevant (lexical match), but user did

Note: This is noisy! App_B might be relevant but just wasn't seen (posi

Mitigation: Only use apps that were in top-3 positions but not clicked.

Method 2: In-batch negatives + hardest selection

Given a batch of $B=256$ pairs (q_i, d_i^+) :

$$S = \mathbf{Q} \cdot \mathbf{D}^T \in \mathbb{R}^{B \times B}$$

- Diagonal S_{ii} = positive pair similarities
- Off-diagonal $S_{ij}, j \neq i$ = in-batch negatives

$$j_{\text{hardest negative for } q_i} = \arg\max_{j \neq i} S_{ij}$$

These are apps that are someone else's positive but score high for q_i — the model is most confused about them.

Method 3: Same-category but different function

Query: "photo editor"

Positive: Adobe Lightroom (Photo & Video → editing)

Hard negatives:

- Photo Gallery (Photo & Video → viewing, NOT editing)
- Camera+ (Photo & Video → capture, NOT editing)
- Photo Collage Maker (Photo & Video → collage, not general editor)

Label logic: Same category implies surface similarity, but different fu

How to get: category metadata + functionality classification (manual or

Method 4: Model-based iterative mining (ANCE approach)

Round 1:

- Train dual encoder with random negatives
- Use trained model to retrieve top-100 for each training query
- Candidates in top-100 that are NOT positive = hard negatives for ne

Round 2:

- Re-train with these hard negatives
- Model improves → old hard negatives become easy
- Mine new hard negatives from updated model's top-100

Round 3:

- Repeat (usually 2-3 rounds is enough, diminishing returns after)

Key insight: Hard negative is a SAMPLING strategy, not a labeling strategy

You don't need new labels for hard negatives! The label logic is simple:

- If (query, app) pair has a click/download → positive (regardless of difficulty)
- If (query, app) pair has NO click/download → negative
- "Hard" just means: among all negatives, we SELECT the ones the model finds confusing

The "hardness" is about WHERE you sample negatives from, not about getting new labels:

- Random sampling → easy negatives (from entire corpus)
- BM25 top results → medium-hard negatives (lexically similar)
- Model's own top results → hardest negatives (semantically similar to model)

Practical training recipe:

For each query \$q\$, construct 1 positive + mixed negatives:

Negative source	Count	Difficulty
Random from corpus	~5	Easy (baseline contrast)
BM25 top results not clicked	~3	Medium (lexical confusion)
In-batch (other queries' positives)	\$B-1\$	Variable (free)
Model-mined from previous checkpoint	~2	Hardest (optional)

$$\mathcal{L} = -\log \frac{\exp(\mathbf{q}^T \mathbf{d} + \tau)}{\exp(\mathbf{q}^T \mathbf{d} + \tau) + \sum_{i=1}^K \exp(\mathbf{q}^T \mathbf{d}_i - \tau)}, \quad \tau = 0.07$$

How to say it in interview (Hard Negative Mining):

"Hard negatives are samples that aren't relevant but LOOK relevant to the model — they have high similarity scores despite being negative. They're critical because with only easy random negatives, the model quickly achieves near-perfect accuracy, the loss goes to zero, and it stops learning. It can't distinguish 'photo editor' from 'photo gallery' because both score high.

I'd source hard negatives from three places: BM25 top results that weren't clicked — these are things the lexical system thinks are relevant but users disagreed. In-batch negatives that happen to score high against the current query. And iteratively from the model itself — after training one round, use the model to retrieve its top results, and the ones that are actually negative become the hardest negatives for the next round. This is the ANCE approach.

The key insight: hard negative mining is a sampling strategy, not a labeling strategy. The labels don't change — positive is still clicked/installed, negative is still not. We're just choosing WHICH negatives to show the model during training to maximize learning signal."

Q7d: Why can relevance model use cross-encoder? Isn't dual encoder

better?

Clarification: Cross-encoder quality > Dual encoder quality

Common misconception: Dual encoder is "better." In fact:

	Dual Encoder	Cross-Encoder
Quality	Lower (no cross-attention)	Higher (full cross-attention)
Speed	Extremely fast (ANN)	Slow (per-pair inference)
Use case	Retrieval (50K candidates)	Ranking/filtering (100-500 candidates)

Dual encoder's advantage is **speed**, not quality. It sacrifices quality for the ability to do ANN search over 50K+ candidates.

Why retrieval MUST use dual encoder:

- Must search 50K candidates → can't run model on each pair
- Must pre-compute doc embeddings offline + ANN lookup
- Only dual encoder allows this (independent encoding of query and doc)

Why relevance model CAN use cross-encoder:

- It runs **after** retrieval, candidates already reduced to ~100-500
- 100 pairs × 5ms/pair = 500ms — expensive but feasible
- Since candidate count is small, we can afford a heavier, more accurate model

The multi-stage principle:

50K candidates → Dual encoder (fast, moderate quality)
500 candidates → Cross-encoder relevance (slow, high quality)
100 candidates → Ranking model (feature-rich)

Each stage: fewer candidates → heavier model → higher quality.

How to say it in interview (Why cross-encoder for relevance):

"Actually, the cross-encoder has HIGHER quality than the dual encoder — not lower. The dual encoder's only advantage is speed. It encodes query and document independently so we can pre-compute document embeddings and do ANN search over 50K candidates. But that independence means it can't model fine-grained interactions between query and document tokens.

A cross-encoder concatenates query and document and passes them through the same transformer — every token can attend to every other token. So it captures nuances like 'photo editor' is NOT the same as 'photo gallery,' even though they share the word 'photo.'

The reason we can use it for the relevance model is simple: it runs AFTER retrieval, when candidates are already reduced to a few hundred. At that scale, per-pair inference is affordable. So the multi-stage design is: dual

encoder for speed at the top of the funnel, cross-encoder for quality at the bottom."

Q7e: Is cross-encoder just pointwise ranking?

Yes, essentially pointwise.

$f(q, d) = \text{Transformer}([\text{CLS}]; q; [\text{SEP}]; d; [\text{SEP}])$
to $\text{score} \in [0, 1]$

- Input: one (query, candidate) pair concatenated
- Output: single relevance score
- Scores each pair independently → pointwise

Key difference from dual encoder:

- **Dual encoder:** query and doc go through **separate** encoders, tokens never see each other, only interact via final dot product
- **Cross-encoder:** query and doc tokens enter **same** transformer, every token attends to all other tokens (full cross-attention)

This is why cross-encoder captures fine-grained interactions:

- "photo editor" vs "Photo Gallery" — cross-encoder understands "editor" ≠ "gallery" via cross-attention
- Dual encoder may encode both similarly (because "photo" dominates the embedding)

How to say it in interview (Cross-encoder = pointwise):

"Yes, a cross-encoder is essentially pointwise scoring. You concatenate the query and document with separator tokens, pass it through a transformer, and the CLS token output gives you a single relevance score. It scores each query-document pair independently. The key difference from a dual encoder is that here the query and document tokens can attend to each other inside the transformer — that full cross-attention is what gives it higher quality at the cost of needing per-pair inference."

Q7f: If retrieval already has multi-objective loss, why need a separate relevance model?

Core reason: Multi-objective is a compromise, not best-of-both

$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{click}} + \beta \cdot \mathcal{L}_{\text{download}} + \gamma \cdot \mathcal{L}_{\text{relevance}}$

One embedding space tries to satisfy three conflicting objectives:

- A popular app (high click) may be irrelevant to the specific query
- A highly relevant app may have low CTR (unknown app)

One embedding cannot perfectly satisfy all objectives. Result: each objective is diluted. Especially when click data (α) is 100x more abundant than relevance data (γ), the relevance signal gets drowned out.

Advantages of a standalone relevance model:

1. **No compromise** — 100% model capacity dedicated to relevance alone
2. **Stronger architecture** — not constrained by ANN, can use cross-encoder
3. **Hard gate** — set threshold: $\text{relevance} < 0.3 \implies \text{filter out}$, regardless of revenue
4. **Independent iteration** — update retrieval and relevance models separately

How they complement each other:

Retrieval (dual encoder, multi-obj):

- Coarse filter, ensures relevant + high-revenue candidates recalled
- Relevance judgment is rough (one embedding for all objectives)

Relevance model (standalone):

- Precise relevance judgment on smaller candidate set
- Acts as safety net against irrelevant-but-popular apps

Ranking:

- Among "confirmed relevant" candidates, maximize revenue

Multi-objective retrieval ensures "don't completely miss relevant items," but its relevance is coarse. The standalone model provides fine-grained precision.

How to say it in interview (Why separate relevance model):

"Even with a multi-objective retrieval model, I'd add a standalone relevance model. The reason is that multi-objective training is fundamentally a compromise — one embedding space trying to satisfy click, download, and relevance objectives simultaneously. These objectives conflict: a popular app gets high click scores but might be irrelevant to the specific query. And since click data is 100x more abundant than relevance labels, the relevance signal gets diluted.

A standalone relevance model dedicates 100% of its capacity to one question: is this app relevant to this query? It acts as a hard quality gate — if relevance is below threshold, the candidate is dropped regardless of revenue potential. This way the retrieval model can focus on broad recall, and the relevance model provides precision as a safety net."

Q7g: What is distillation? Why not just use cross-encoder directly?

Problem: Cross-encoder is too slow for online serving

Scenario

Latency

500 candidates $\times$ 5ms/pair 2.5 seconds ✗
 100 candidates $\times$ 5ms/pair 500ms (borderline)
 User expectation < 100ms total

Solution: Knowledge Distillation

$\text{Teacher (cross-encoder)} \xrightarrow{\text{distill}} \text{Student (lightweight model)}$

Step 1: Train cross-encoder on $\sim 50\text{K}$ human-labeled (query, app, relevance) pairs

- Works with small data because pre-trained LM (BERT) already understands semantics
- Fine-tuning just teaches it "what counts as relevant in ads context"

Step 2: Use cross-encoder to score millions of (query, app) pairs **offline**

- No latency constraint — can run for days
- Produces high-quality soft labels: e.g., $\text{score}(\text{"photo editor"}, \text{App_A}) = 0.92$

Step 3: Train student model on these soft labels

- Student: 2-layer MLP, small dual encoder, or GBDT
- $\mathcal{L} = \text{MSE}(\text{student}(q, d), \text{teacher_score}(q, d))$
- Now you have millions of high-quality labels to train with

Step 4: Serve student model online

- Latency: $\sim 0.1\text{-}0.5\text{ms/pair} \rightarrow 500 \times 0.1\text{ms} = 50\text{ms} \checkmark$

Comparison:

	Cross-encoder (teacher)	Distilled student
Quality	Highest	$\sim 95\%$ of teacher
Latency	5-10ms/pair	0.1-0.5ms/pair
500 candidates	2.5-5s ✗	50-250ms $\checkmark$
Use case	Offline scoring / labeling	Online serving

How to say it in interview (Distillation):

"The cross-encoder gives the highest quality relevance scores, but at 5ms per pair on 500 candidates, that's 2.5 seconds — way too slow for real-time serving. So we use knowledge distillation: train the cross-encoder offline on human-labeled data, then use it to score millions of query-app pairs offline — no latency constraint, can run for days. Those scores become high-quality soft labels. Then we train a lightweight student model — a small MLP or GBDT — on those millions of labels. The student achieves about 95% of the teacher's quality but runs in 0.1ms per pair. At serving time, 500 candidates times 0.1ms equals 50ms — totally acceptable."

Q7h: Why not just train GBDT directly? Why distill?

The label quality/quantity problem

Option: Train GBDT directly

Label source	Volume	Problem
Human annotation	50K-200K	Too few for GBDT to generalize well
Click/download logs	Millions	Noisy; click $\neq$ relevance

GBDT is a shallow model — it doesn't have pre-trained language understanding. It needs **large amounts of clean labels** to learn well.

Option: Distill from cross-encoder

Step	What happens	Volume
50K human labels $\rightarrow$ cross-encoder	BERT already "understands" language, 50K is enough to fine-tune	50K
Cross-encoder $\rightarrow$ score 5M pairs	High-quality soft labels generated offline	5M
5M soft labels $\rightarrow$ GBDT	Abundant + clean training data	5M

Why cross-encoder works with only 50K labels but GBDT doesn't:

Cross-encoder = BERT + fine-tuning:

- BERT is pre-trained on billions of tokens — already knows "editor" $\neq$ "viewer"
- Fine-tuning only teaches: "in ads context, what specific patterns = relevant"
- 50K examples is plenty for this narrow fine-tuning

GBDT = no language prior:

- Operates on hand-crafted features: BM25 score, category match, cosine sim, etc.
- Has no concept of word meaning — just splits on numeric thresholds
- 50K examples $\rightarrow$ learns shallow decision boundaries, overfits or misses edge cases
- Needs millions of examples to cover the space of (query, app) combinations

Cross-encoder as a "label amplifier":

$50\text{K human labels} \rightarrow \text{cross-encoder} \rightarrow 5\text{M high-quality soft labels} \rightarrow \text{train} \rightarrow \text{GBDT}$

The cross-encoder multiplies your expensive human labels 100x. GBDT couldn't learn from 50K, but it CAN learn from 5M.

When CAN you skip distillation and train GBDT directly?

- You have millions of clean relevance labels (e.g., massive annotation budget)
- Your features are already highly informative (e.g., includes a pre-computed embedding similarity from another model)
- Latency is not a constraint (can use cross-encoder directly)

Is dual encoder train/serve inconsistent?

Not really. Both training and serving use the same operation:

$$\text{score}(q, d) = \mathbf{q}^T \mathbf{d}$$

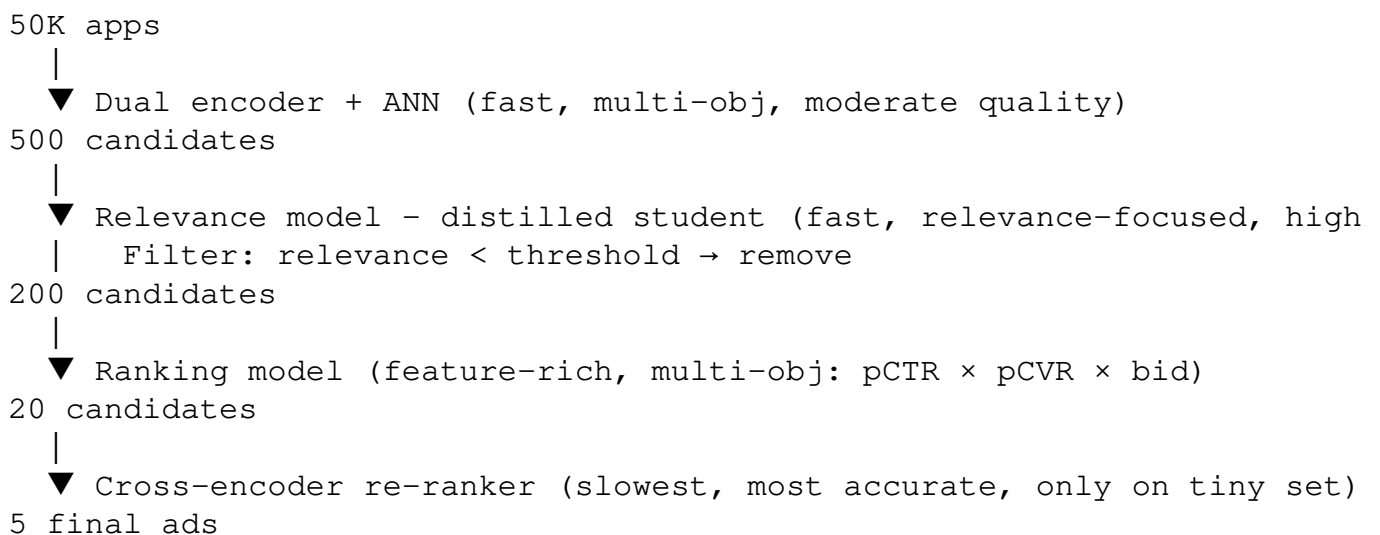
- Training: compute $\mathbf{q}^T \mathbf{d}$ directly, apply InfoNCE loss
- Serving: compute $\mathbf{q}^T \mathbf{d}$ via ANN (approximate, ~95-99% recall vs exact)

ANN is not a "proxy" — it's an **acceleration of exact dot product search**. Like binary search is an acceleration of linear search; same answer, faster.

The real train/serve gap is in negative distribution:

- Training: model sees in-batch negatives + sampled hard negatives (hundreds)
- Serving: model must distinguish among 50K real candidates
- Mitigation: better hard negative mining bridges this gap

Full system model design logic:



Design principle: fewer candidates → heavier model → higher quality → more latency budget per candidate.

How to say it in interview (Why not train GBDT directly):

"The core issue is label quality and quantity. GBDT is a shallow model with no language understanding — it operates on hand-crafted features like BM25 score, category match, embedding similarity. With only 50K human-labeled pairs, it overfits and can't generalize across the huge space of query-app combinations.

But a cross-encoder, which is BERT-based, already 'understands' language from pre-training. It knows that 'editor' and 'viewer' are different functions without being explicitly taught. Fine-tuning on 50K examples is plenty — you're just teaching it what 'relevant' means in your specific ads context.

So the cross-encoder acts as a label amplifier: it takes 50K expensive human labels, generates 5 million high-quality soft labels by scoring pairs offline, and then those 5 million labels are enough for the GBDT to learn well. You couldn't skip the cross-encoder because GBDT needs millions of clean labels that you can't get directly from humans at reasonable cost."

How to say it in interview (Cross-encoder vs Dual encoder, Relevance model, Distillation — complete story):

"A common misconception is that dual encoders are higher quality than cross-encoders — it's actually the opposite. A cross-encoder gets full cross-attention between query and document tokens, so it captures fine-grained interactions that a dual encoder misses. The dual encoder's advantage is purely speed: it lets us pre-compute document embeddings and do ANN search.

So the architecture follows a principle: as we move down the funnel and candidates decrease, we use progressively heavier models. Retrieval must use a dual encoder because we're searching 50K candidates. But once we're down to 500 candidates, we CAN afford a cross-encoder — and we should, because it's more accurate.

A cross-encoder is essentially pointwise scoring: concatenate query and document, pass through a transformer, get a relevance score. It's like a binary classifier for each pair.

Now, why a separate relevance model even if our retrieval model already has a relevance objective? Because multi-objective training is a compromise. One embedding space trying to satisfy click, download, AND relevance objectives simultaneously — they conflict. A popular app might get high click scores but be irrelevant to the specific query. The relevance signal gets diluted, especially since click data is 100x more abundant. A standalone relevance model dedicates 100% of its capacity to one job: is this app relevant to this query? It acts as a hard quality gate.

For serving this relevance model, the challenge is latency. A cross-encoder takes 5ms per pair — on 500 candidates that's 2.5 seconds, way too slow. So we use knowledge distillation: train the cross-encoder offline on 50K human-labeled pairs — BERT's pre-trained understanding makes this work with limited data. Then use it to score millions of pairs offline, generating high-quality soft labels. Finally, train a lightweight student model — a small MLP or GBDT — on those millions of soft labels. The student achieves about 95% of the teacher's quality but runs in 0.1ms per pair.

Why not skip the cross-encoder and train the GBDT directly? Because GBDT has no language understanding — it operates on hand-crafted features. With only 50K human labels it overfits. But BERT already 'knows' that editor and viewer are different functions — 50K fine-tuning examples is plenty to teach it

what 'relevant' means in ads context. Then it amplifies those 50K labels into 5 million high-quality training samples for the GBDT. The cross-encoder is essentially a label amplifier."

Retrieval Model Training — Connecting the Dots (Complete Summary)

Everything about training a retrieval dual encoder connects into one coherent story:

1. TRAINING DATA & ITS PROBLEMS

Sources:

- Install logs (strong positive, sparse)
- Click logs (weak positive, abundant, noisy)
- Human annotation (strong, small scale)
- LLM-as-judge labels (strong, scalable)

Fundamental problem: all behavioral labels are BIASED

- Position bias: clicks confounded by display position
- Popularity bias: popular apps get clicked regardless
- Fix: IPW (upweight low-position clicks), use installs > clicks

2. NEGATIVE SAMPLING (what to contrast against)

Easy negatives: random from corpus (model learns fast, then stops)

Medium: BM25 top not clicked (lexically similar but not relevant)

Hard: in-batch negatives (other queries' positives)

Hardest: model-mined (ANCE – model's own top false positives)

Also a sampling strategy, not a labeling strategy.

The label is always: clicked/installed = positive, else = negative

"Hard" = WHERE you sample, not WHAT the label is

3. LOSS FUNCTION (how to learn from the data)

InfoNCE (contrastive): softmax over similarities

- 1 positive + K negatives → "which is the real match?"
- Weighted InfoNCE: download pairs get 2x gradient weight
- Alternative: Cosine MSE with soft labels (graded relevance)

Negative sampling fills the denominator of InfoNCE

Harder negatives → more informative gradients

4. MULTI-OBJECTIVE (what to optimize for)

Single embedding must satisfy multiple objectives:

$$L_{\text{total}} = \alpha \cdot L_{\text{click}} + \beta \cdot L_{\text{download}} + \gamma \cdot L_{\text{relevance}}$$

Problem: objectives conflict. Revenue vs relevance.

One embedding = compromise. Relevance gets diluted.

5. STANDALONE RELEVANCE MODEL (dedicated quality gate)

Why: multi-obj retrieval can't do relevance perfectly

Architecture: cross-encoder (higher quality, runs on fewer cands)

Labels: human annotation (50K) or LLM-as-judge

Serving: too slow directly → distill to lightweight student

6. DISTILLATION (making it servable)

Teacher: cross-encoder trained on 50K human labels (BERT handles small data via pre-training)

→ Score 5M pairs offline → high-quality soft labels

Student: GBDT/MLP trained on 5M soft labels → serves in 0.1ms

Cross-encoder = "label amplifier" (50K → 5M clean labels)

How each piece answers "why do we need the next one?":

1. **Data has bias** → need IPW / careful negative selection
2. **Easy negatives aren't enough** → need hard negative mining
3. **Multiple objectives conflict** → need standalone relevance model
4. **Cross-encoder too slow** → need distillation
5. **GBDT can't learn from 50K labels** → need cross-encoder as label amplifier

How to say it in interview (complete retrieval training story, 2 minutes):

"Let me walk through how I'd train a retrieval dual encoder end-to-end.

First, training data. Positive labels come from search logs — installs are strong positives but sparse, clicks are weaker but abundant. The key challenge is that this data has position bias: apps in top positions get clicked more just because they're seen, not because they're more relevant. I'd address this with inverse propensity weighting — upweighting clicks from lower positions since those indicate stronger user intent.

For negatives, I'd use a mix: random corpus samples as a baseline, BM25 top results that weren't clicked as medium-hard negatives, in-batch negatives for free, and potentially model-mined hard negatives from a previous training checkpoint. Hard negatives are critical — without them the model converges too fast on easy distinctions and can't differentiate 'photo editor' from 'photo gallery.'

The loss is InfoNCE — a softmax over similarities. One positive plus K negatives, maximize the probability of selecting the correct positive. I'd weight it so download pairs contribute 2x the gradient compared to click pairs.

For multiple objectives — click, download, and relevance — I'd add them as separate loss terms on the same embedding space. But this is inherently a

compromise: one embedding trying to satisfy conflicting goals means each gets diluted.

That's why I'd add a standalone relevance model downstream — a cross-encoder that dedicates 100% capacity to relevance alone, acting as a hard quality gate. For serving latency, I'd distill it: train the cross-encoder on 50K human labels, use it to score millions of pairs offline, then train a lightweight student on those soft labels. The cross-encoder works as a label amplifier — BERT's pre-trained knowledge makes 50K labels enough, then it generates millions of clean training samples for the fast student model."

Q8: ANN (Approximate Nearest Neighbor) — Why Fast, How It Works

Why exact KNN is slow:

$O(N \times d)$ distance computations per query

- 50K docs $\times$ 256 dims = 12.8M multiplications per query
- For millions of docs $\rightarrow$ completely infeasible at serving latency

Core idea of ANN:

Don't look at ALL candidates. Only examine a small subset that's likely to contain the true nearest neighbors. Trade a tiny recall loss ($\sim 1-5\%$) for orders-of-magnitude speedup.

Method 1: IVF (Inverted File Index) — Cluster-based

Offline — Build Index:

1. K-means cluster all N doc embeddings into C clusters (e.g., $C=1000$)
2. Each cluster has a **centroid** (the "mother node")
3. Record which docs belong to each cluster

```
Cluster 0 (centroid_0): [doc_23, doc_156, doc_892, ...]
Cluster 1 (centroid_1): [doc_7, doc_44, doc_501, ...]
...
Cluster 999 (centroid_999): [doc_12, doc_88, ...]
```

Online — Query:

Step 1: Compare query to all $C=1000$ centroids
→ Find $n_{\text{probe}}=10$ nearest clusters
Cost: $O(C \times d) = 1000 \times 256 = \text{fast!}$

Step 2: Only search docs INSIDE those n_{probe} clusters
Each cluster has $\sim N/C = 50000/1000 = 50$ docs
Search volume: $n_{\text{probe}} \times (N/C) = 10 \times 50 = 500$ distance computat

Step 3: Return top-K from those 500 candidates

Speedup: Was 50,000 distance computations → now 1000 (centroids) + 500 (within clusters) = 1,500. ~33x faster.

Key parameters:

Parameter	What it controls	Trade-off
C (nlist)	Number of clusters	More clusters → faster search, but may miss boundary items
n_{probe}	How many clusters to search	Higher → better recall, slower

$\text{Recall} \uparrow \text{ as } n_{probe} \uparrow, \quad \text{Latency} \uparrow \text{ as } n_{probe} \uparrow$

Method 2: HNSW (Hierarchical Navigable Small World) — Graph-based

Completely different approach: build a **multi-layer graph** (like a skip list in vector space).

Offline — Build Graph:

Each doc embedding = a node in the graph. Each node connects to M nearest neighbors. Multiple layers with decreasing density:

Layer 3 (sparsest): Few nodes, long-range connections
[node_A] ————— [node_B] ————— [node_C]

Layer 2: More nodes, medium-range connections
[A] — [D] — [B] — [E] — [C]

Layer 1: Even more nodes, short-range connections
[A] - [F] - [D] - [G] - [B] - [H] - [E] - [I] - [C]

Layer 0 (densest): ALL nodes present, each connected to M nearest neighbors
All 50,000 nodes, each with $M=32$ edges to nearest neighbors

Online — Greedy Graph Traversal:

Step 1: Start at entry point on Layer 3 (sparsest)
Greedy jump to whichever neighbor is closest to query
Until no neighbor is closer (local minimum on this layer)

Step 2: Drop to Layer 2, continue greedy search from that position
Finds a more precise neighborhood

Step 3: Drop to Layer 1 → continue

Step 4: On Layer 0, do fine-grained search
Maintain a priority queue of top-K candidates

Explore neighbors-of-neighbors until convergence

Analogy — finding a restaurant on a map:

- Layer 3: Which city? (big jumps)
- Layer 2: Which district? (medium jumps)
- Layer 1: Which street? (small jumps)
- Layer 0: Exact address (fine search)

Complexity:

$\text{HNSW search} \approx O(\log N \times M \times d)$

For $N=50,000$: $\log N \approx 17$, so $\sim 17 \times 32 \times 256 \approx 139\text{K}$ operations vs 12.8M for exact. $\sim 100\text{x}$ faster.

Key parameters:

Parameter	What it controls	Trade-off
M	Neighbors per node	Higher $\rightarrow$ better recall, more memory
ef_construction	Beam width when building graph	Higher $\rightarrow$ better graph quality, slower build
ef_search	Beam width at query time	Higher $\rightarrow$ better recall, slower query

Method 3: IVF-PQ (Cluster + Compression)

When N is very large (millions+), storing all full vectors in memory is expensive. **Product Quantization** compresses each vector:

$\mathbf{d} \in \mathbb{R}^{256} \rightarrow [c_1, c_2, \dots, c_8], \quad c_i \in \{0, \dots, 255\}$

Split 256-dim vector into 8 sub-vectors of 32 dims each. Quantize each sub-vector to its nearest centroid (from a 256-entry codebook).

	Raw vector	PQ-compressed
Storage	$256 \times 4 \text{ bytes} = 1024 \text{ bytes}$	8 bytes (one uint8 per segment)
Compression	1x	128x

Distance computation uses lookup tables (pre-computed partial distances) $\rightarrow$ still fast.

IVF vs HNSW Comparison

	IVF	HNSW
Idea	Cluster vectors, only search nearby clusters	Navigate multi-layer graph greedily

Build	K-means clustering	Insert nodes one by one, connect edges
Memory	Low (vectors + cluster IDs)	High (vectors + full graph structure)
Query speed	Fast	Faster (especially at high recall)
Recall/speed	Good	Best
Parameters	nlist, nprobe	M, ef_search
Best for	Ultra-large scale (billions)	Medium-large scale (millions)

FAISS Code Examples with Parameters Explained

```
import faiss

d = 256 # embedding dimension

# === Option 1: HNSW (best recall/speed for <10M items) ===
index = faiss.IndexHNSWFlat(d, M=32)
# M=32: each node connects to 32 neighbors
# Larger M → higher recall, more memory (~50K × 32 × 4 bytes for edges)

index.hnsw.efSearch = 64 # beam width at query time
# Larger ef → higher recall, slower
# Typical: ef=64 gives ~99% recall, ef=128 gives ~99.5%

index.add(app_embeddings) # shape: (50000, 256)

# Query
distances, ids = index.search(query_emb, k=500)

# === Option 2: IVF-Flat (cluster-based, exact within clusters) ===
nlist = 1000 # number of clusters
quantizer = faiss.IndexFlatL2(d) # used to find nearest clusters
index = faiss.IndexIVFFlat(quantizer, d, nlist)

index.train(app_embeddings) # train K-means to find centroids
index.add(app_embeddings)

index.nprobe = 10 # search 10 nearest clusters at query time
# nprobe=1: fastest, ~60% recall
# nprobe=10: good balance, ~95% recall
# nprobe=100: near-exact, ~99.5% recall, slower

distances, ids = index.search(query_emb, k=500)
```



```
# === Option 3: IVF-PQ (cluster + compression, for large scale) ===
index = faiss.IndexIVFPQ(quantizer, d, nlist=1000, m=8, nbits=8)
# m=8: split vector into 8 sub-vectors
# nbits=8: 2^8=256 centroids per sub-vector
# Memory per vector: 8 bytes instead of 1024 bytes

index.train(app_embeddings)
index.add(app_embeddings)
index.nprobe = 20 # may need higher nprobe to compensate for PQ approx
```

Industrial workflow:

Offline:

1. Compute all app embeddings using app encoder
2. Build ANN index (FAISS HNSW for <10M, IVF-PQ for >10M)
3. Deploy index to serving infra (replicated for availability)
4. Periodically rebuild as new apps are added

Online (per query):

1. Encode query → 256-dim embedding (~5ms on GPU, ~20ms on CPU)
 2. ANN search → top-K candidate IDs (~2-5ms)
 3. Fetch candidate metadata → pass to ranker
-

How to say it in interview (ANN — full version):

"Exact nearest neighbor search is $O(N \times d)$ — you compute the distance between the query embedding and every document embedding. For 50K items with 256 dimensions, that's about 13 million multiplications per query. It's feasible at 50K, but not at millions.

ANN avoids exhaustive search. There are two main approaches. The first is IVF — Inverted File Index. You offline cluster all document embeddings into, say, 1000 clusters using K-means. At query time, you first find the 10 nearest cluster centroids — that's 1000 distance computations instead of 50,000. Then you only search documents within those 10 clusters. If each cluster has about 50 documents, you're searching 500 documents total instead of 50,000. Roughly 33x speedup with maybe 95% recall.

The second approach is HNSW — Hierarchical Navigable Small World graphs. You build a multi-layer graph where each node is a document and each node connects to its M nearest neighbors. Top layers are sparse with long-range connections, bottom layers are dense with local connections. At query time, you start at the top layer and greedily hop to whichever neighbor is closest to your query. Then you drop to the next layer and repeat. It's like a skip list but in vector space — you do big jumps first to get to the right neighborhood, then fine-grained search at the bottom. This gives $O(\log N)$ search complexity instead of $O(N)$.

For our scale of 50K-10M items, HNSW typically gives the best recall-speed tradeoff. For billions-scale, IVF with Product Quantization compresses each vector from 1024 bytes to 8 bytes — a 128x memory reduction — which makes it feasible to fit everything in RAM."

How to say it (30-second version):

"ANN avoids exhaustive search via two main approaches. IVF clusters vectors offline and only searches nearby clusters at query time — reducing search space from N to $n_{\text{probe}} \times (N/n_{\text{list}})$. HNSW builds a multi-layer navigable graph where you greedily traverse from coarse to fine layers, like a skip list in vector space. Both trade $\sim 1\text{-}5\%$ recall for 10-100x speedup. HNSW gives the best recall-speed tradeoff for moderate scale (up to $\sim 10\text{M}$); IVF-PQ is better for billions-scale with memory constraints."

Q8b: Multi-Channel Retrieval — Strategies Beyond BM25 + Semantic

All retrieval channels in a production ads system:

Channel	Method	Strength	Covers what
Lexical	BM25 / inverted index	Exact term matching, no training	Query $\leftrightarrow$ app name exact match
Semantic	Dual encoder + ANN	Understands synonyms/paraphrases	"photo editor" $\rightarrow$ "image retouching tool"
Offline pre-computed	Query $\rightarrow$ app lookup table	Zero latency, can use heavy models	High-frequency queries' known-best results
Graph-based	Co-click / co-install graph	Captures behavioral patterns	"Users who installed A also installed B"
Rule-based	Exact name match, category filter	100% precision for navigational queries	"Instagram" $\rightarrow$ return Instagram directly

Offline Pre-computed Retrieval — What, How, Why

What is it?

Pre-compute "query $X \rightarrow$ candidate apps" mapping **offline**, store as a lookup table. At serving time: query comes in $\rightarrow$ table lookup $\rightarrow$ 0ms retrieval latency.

Offline lookup table:

```
"photo editor"       $\rightarrow$  [Lightroom, VSCO, Snapseed, Pixlr, ...]
"video editor"       $\rightarrow$  [CapCut, iMovie, InShot, ...]
"fitness tracker"    $\rightarrow$  [Nike Run, Strava, Fitbit, ...]
...
```

How to build the offline dataset:

Method 1: From historical engagement (click/download logs)

Step 1: Aggregate (query, app, engagement_count) from search logs
Step 2: For each query, rank apps by engagement, take top-K
Step 3: Store as lookup table, refresh daily/weekly

Example:

"photo editor" → apps clicked in past 30 days:
Lightroom (50,000 clicks)
VSCO (35,000 clicks)
Snapseed (28,000 clicks)

Method 2: Using heavy model scoring offline

Step 1: Identify high-frequency queries (top-10K queries ≈ 80% traffic)
Step 2: For each query, run cross-encoder against ALL 50K apps
Cost: $50K \times 5ms = 250s$ per query (offline, acceptable)
 $10K \text{ queries} \times 250s = \sim 29 \text{ days}$ → use GPU parallelism → feasible
Step 3: Store top-500 per query in table

Benefit: Cross-encoder quality at zero serving latency

Method 3: Co-occurrence graph expansion

Step 1: Build app-app co-install graph
Edge(A, B) weighted by: #users who installed both A and B
Step 2: For each query, identify "seed apps" (known relevant from logs)
Step 3: From seeds, expand via graph neighbors (1-2 hop random walk)
Step 4: Expanded apps become candidates

Example:

"photo editor" → seed: Lightroom
Lightroom co-install neighbors: VSCO, Darkroom, Affinity Photo
→ These become candidates (even if name doesn't contain "photo editor")

Method 4: Query clustering + sharing

Step 1: Cluster similar queries by embedding similarity
Cluster: {"photo editor", "image editor", "pic editing app", "e
Step 2: Merge engagement data across all queries in same cluster
Step 3: Share candidate list for all queries in cluster

Benefit: Solves data sparsity for low-frequency queries

Advantages of offline retrieval:

Advantage	Why
Zero latency	Pure table lookup, no real-time inference
Can use heaviest models	Offline → no latency budget → cross-encoder, LLM, anything

- App A: rank 1 in BM25, rank 3 in semantic, rank 5 in offline

$$\text{RRF}(A) = \frac{1}{60+1} + \frac{1}{60+3} + \frac{1}{60+5} = 0.0164 + 0.0159 + 0.0154 = 0.0477$$

- App K: rank 2 in graph-based only

$$\text{RRF}(K) = \frac{1}{60+2} = 0.0161$$

App A scores much higher because it appears in 3 channels → more confident.

Why RRF works well:

- No need to normalize different score ranges (BM25 scores vs cosine scores vs ranks)
- Only uses rank → robust to score scale differences
- Simple, no training needed
- Industry standard (Elasticsearch uses it for hybrid search)

Approach C: Pre-ranking model (lightweight learned fusion)

Union (~1200 candidates)

↓

Pre-ranker: 2-layer MLP or lightweight model

Input features: [bm25_score, semantic_score, offline_rank, channel_co

Output: pre_rank_score

Latency: ~0.05ms per candidate → 1200 × 0.05ms = 60ms

↓

Top-500 → pass to heavy ranking model

Where does the relevance model fit?

Relevance model ≠ fusion mechanism. It's a separate quality gate:

Union (1200) → Fusion/Pre-ranking (500) → Relevance filter → Ranking →

↑

Here: "Is this candidate actually relevant to the query, regardless of which channel it came from?"

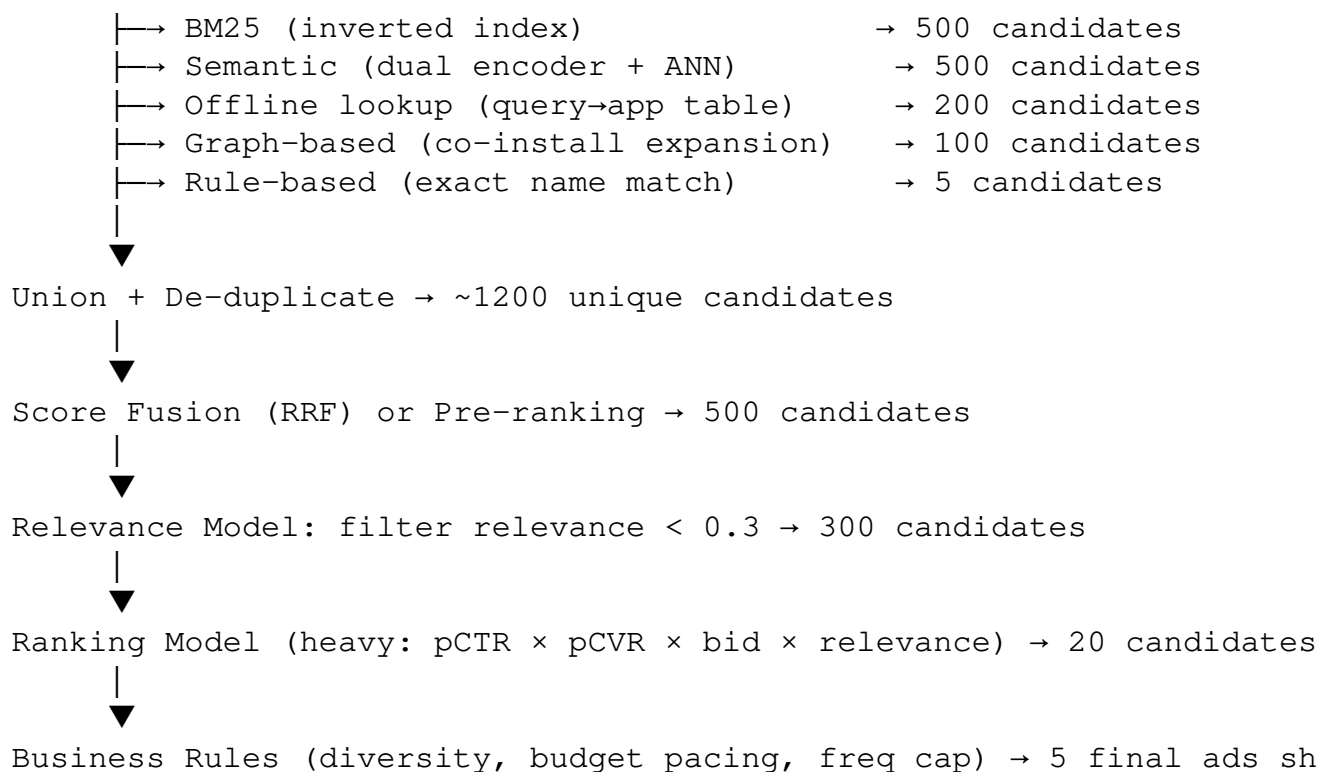
If relevance < threshold → discard
Even if relevance score is high

The relevance model doesn't care about "which channel produced this candidate." It answers one question: **"Given this (query, app) pair, is the app relevant to the query?"**

Complete Multi-Channel Pipeline

User Query: "photo editor"

|



How to say it in interview (Multi-channel retrieval + fusion):

"In production, I wouldn't rely on just one retrieval path. I'd run multiple channels in parallel: BM25 for exact lexical matching, a dual encoder with ANN for semantic matching, an offline pre-computed lookup table for high-frequency queries, and potentially a graph-based channel using co-install signals.

Each channel has different strengths. BM25 handles exact keyword matches perfectly. Semantic retrieval catches synonyms and paraphrases. The offline table can use arbitrarily heavy models like cross-encoders because it's pre-computed — zero serving latency. The graph channel surfaces apps based on behavioral patterns like co-installation.

For the offline table specifically, I'd build it by either aggregating historical engagement data — which query led to which downloads — or by running a cross-encoder offline against all apps for the top 10K highest-frequency queries. Those queries cover maybe 80% of traffic, and the cross-encoder gives the highest quality results with no latency constraint.

To merge results from all channels, I'd use Reciprocal Rank Fusion — RRF. It sums $1 \text{ over } k \text{ plus rank}$ for each channel where the candidate appears. The beauty of RRF is it doesn't need score normalization across channels — it only uses ranks, so you can combine BM25 scores with cosine similarities without calibration. A candidate that appears in multiple channels gets a higher fusion score, which is a strong relevance signal.

After fusion, the relevance model acts as a quality gate — filtering out anything below a relevance threshold regardless of which channel produced

it. Then the ranking model maximizes revenue within the 'confirmed relevant' set."

Q9: Cold-start problem — new app with no behavioral data

1. **Content-based embedding** — encode app title + description + category via the app encoder (it generalizes even without click history since it's trained on text)
2. **Lexical fallback** — BM25 still works for new apps as long as metadata is indexed
3. **Exploration budget** — allocate some impression share to new apps (Thompson sampling or epsilon-greedy)
4. **Transfer from similar apps** — find nearest neighbors by content embedding, use their behavioral statistics as prior
5. **Feature-based warm-up** — use developer reputation, category average CTR, app quality signals as priors in ranker

How to say it in interview:

"For cold-start apps with no behavioral data, the dual encoder actually handles this reasonably well because it's trained on text — a new app's title and description can still produce a meaningful embedding even without click history. BM25 also works immediately since it only needs the app's text to be indexed. For exploration, I'd allocate a small impression budget to new apps — maybe using Thompson sampling to balance exploration and exploitation. And we can borrow behavioral priors from similar existing apps based on embedding proximity or category."

Q10: Query Understanding — From Basic to LLM-based Personalization

Traditional Query Understanding Components

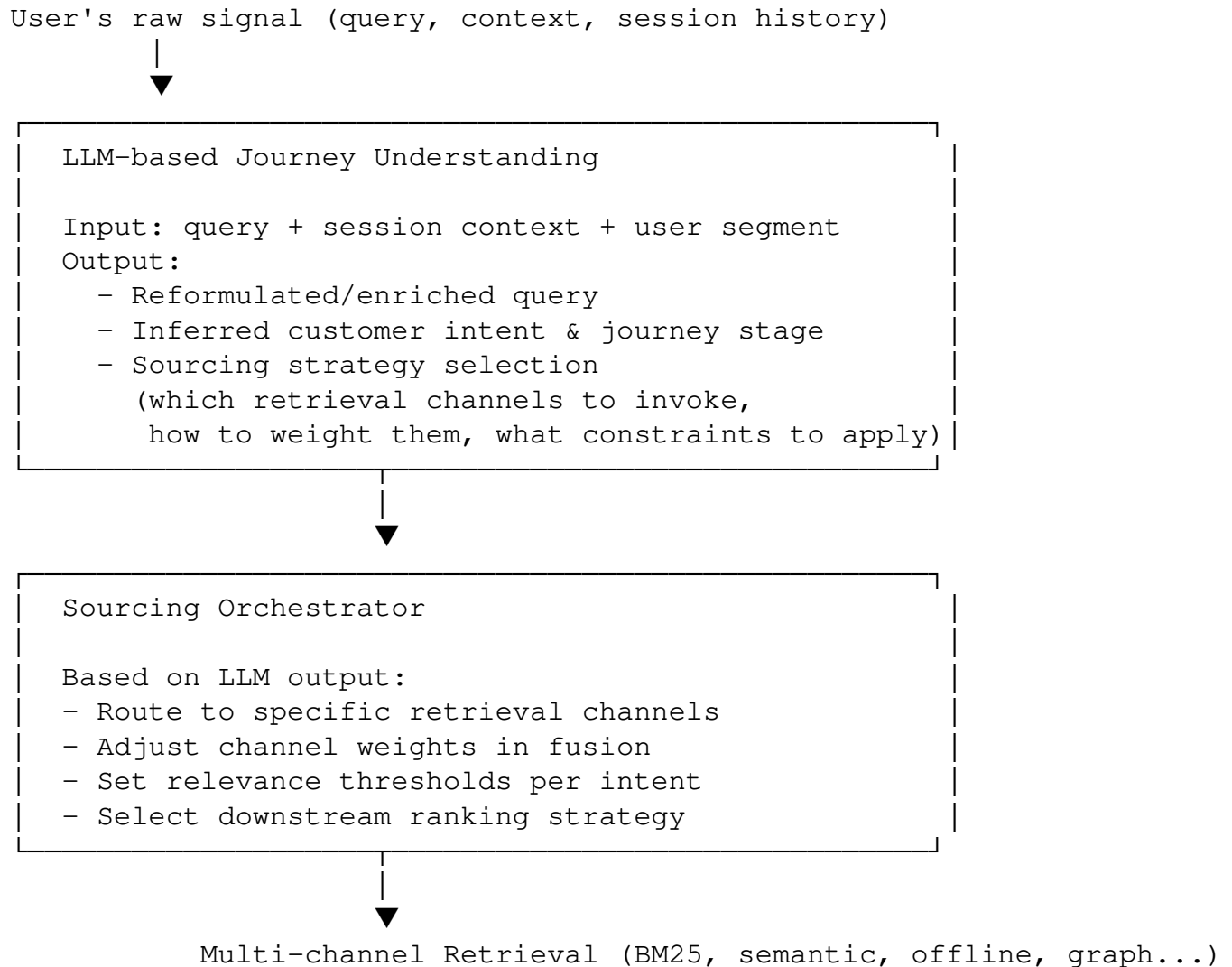
Component	Method	Example
Spell correction	Edit distance + language model	"phot editor" → "photo editor"
Tokenization	WordPiece / custom tokenizer	"photo-editor" → ["photo", "editor"]
Query expansion	Embedding similarity / click graph	"photo editor" + "image editor" + "picture app"
Intent detection	Classifier (navigational vs exploratory)	"Instagram" = navigational; "photo editor" = exploratory
Query rewriting	Seq2seq / LLM	"best free photo app" → "photo editor free"
Entity recognition	NER model	"Adobe Photoshop" → app_name entity

Advanced: LLM-based Customer Journey Understanding + Sourcing Orchestration

Beyond traditional QU, a more advanced approach uses LLMs to understand the customer's

journey and intent in real-time, then dynamically orchestrate sourcing strategies.

Architecture:



Example scenarios:

Journey Stage	LLM Understanding	Sourcing Action
Early exploration	User just browsing "photo apps" broadly	Emphasize diversity, broader semantic retrieval, lower bid threshold
Comparison shopping	User searched "VSCO vs Lightroom"	Return both + similar competitors, weight exact match high
Ready to install	User searched "download Lightroom"	Navigational intent → exact match shortcut, skip broad retrieval
Re-engagement	Returning user, previously installed similar app	Weight graph-based retrieval (co-install), personalized offline table

This IS query understanding — it's the most advanced form of it. Traditional QU components (spell correction, expansion, intent detection) are sub-modules within this broader system. LLM-based journey understanding subsumes and extends all of them:

Traditional QU:

query → spell_correct → expand → detect_intent → fixed retrieval pipe

LLM-based Journey QU:

query + context → LLM understands full customer need →
→ dynamically rewrites query (replaces traditional expansion/rewrit
→ classifies intent (replaces traditional intent classifier)
→ ALSO orchestrates which sourcing strategies to invoke
→ ALSO sets downstream parameters (thresholds, weights, diversity)

Key difference from traditional QU:

- Traditional QU: transforms the query text, then passes to a fixed retrieval pipeline
- LLM journey QU: understands the user's NEED, then orchestrates the entire sourcing strategy dynamically — it controls not just WHAT to search for, but HOW to search

How to say it in interview:

"Query understanding sits between the user and retrieval. At the basic level, it handles spell correction, intent detection, and query expansion. But in my recent work, I took this further with an LLM-based approach to understand the customer's journey in real-time.

The idea is: instead of just fixing typos and expanding terms, we use an LLM to infer the user's actual need and journey stage from the query plus session context. Then based on that understanding, we don't just rewrite the query — we dynamically orchestrate the downstream sourcing strategy. For example, if the LLM detects the user is in early exploration mode, we emphasize diversity and broader semantic retrieval. If they're in comparison mode, we weight exact matches higher and surface competitors. If it's a navigational query, we shortcut directly to exact matching.

This is fundamentally still query understanding — it's just operating at a higher level. Traditional QU transforms the query text. LLM-based journey understanding transforms the entire sourcing strategy based on understanding the user's intent. The LLM subsumes the traditional components — it does expansion, rewriting, and intent detection — but then ALSO controls which retrieval channels to invoke and how to weight them. So it's query understanding that extends into sourcing orchestration."

Q11: Offline vs Online evaluation metrics

Offline metrics (before A/B test):

Metric	Formula	Use case
Recall@K	$\text{relevant} \cap \text{retrieved} / \text{total_relevant}$	Retrieval coverage
Precision@K	$\text{relevant} \cap \text{retrieved} / K$	Retrieval accuracy

MRR	$1/\text{rank_of_first_relevant}$	Speed to first good result
NDCG@K	DCG@K / IDC@K	Ranking quality with graded relevance
Hit Rate	1 if any relevant in top-K, else 0	Basic coverage check

Online metrics (A/B test):

Metric	What it measures	Stakeholder
CTR	Clicks / Impressions	User engagement
CVR	Downloads / Clicks	Conversion quality
Revenue per query	Total ad spend / queries	Business
ROAS	Advertiser revenue / ad spend	Advertiser satisfaction
Relevance (human eval)	Annotated relevance score	Quality
Time to first click	Latency of user decision	UX
Advertiser churn	% advertisers leaving	Long-term health

Q12: LLM applications in ads matching

- Query understanding** — LLM for query expansion, intent classification, reformulation
- Relevance scoring** — LLM-as-judge for query-app relevance (can replace expensive human annotation)
- Embedding models** — LLM-based encoders (E5-large, GTE, BGE) as dual encoder backbone
- Feature extraction** — Extract structured attributes from unstructured app descriptions
- Synthetic data generation** — Generate training pairs for under-represented queries
- Ad creative understanding** — Assess ad copy quality, detect misleading content

Apple privacy constraint on LLMs:

- Cannot fine-tune on individual user data
- Use LLMs for contextual understanding (query-level, app-level) not personalization
- On-device LLM inference possible for some components

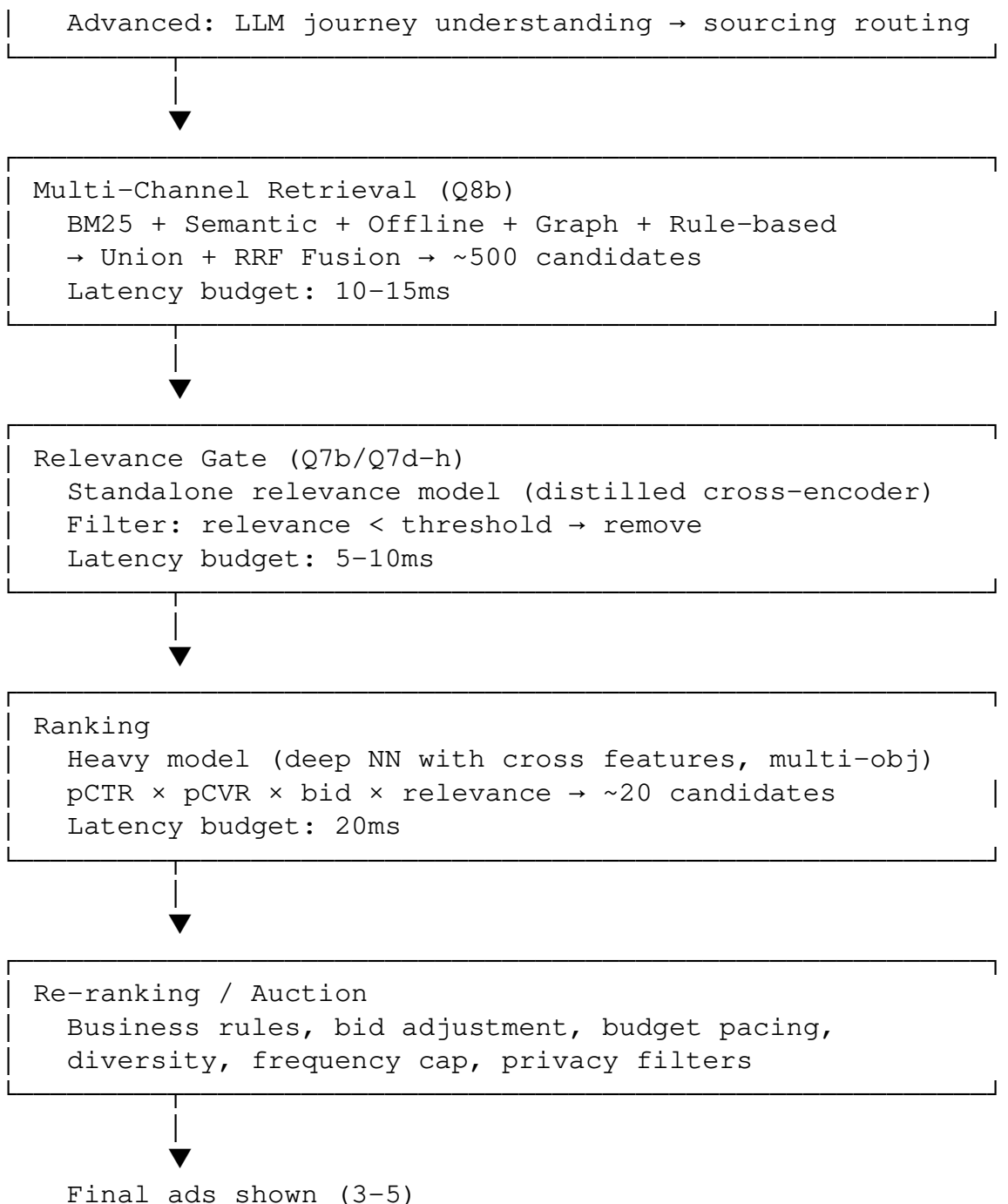
Q13: Multi-stage system architecture

Note: See **Q8b: Multi-Channel Retrieval** for the most complete end-to-end pipeline (multi-channel sourcing → RRF fusion → relevance filter → ranking → auction). The diagram below is a simplified version.

User Query



Query Understanding (Q10)
Traditional: spell correct, expand, intent detect



Key principle: Each stage uses a more complex model on fewer candidates. Total latency budget: ~50-100ms.

How to say it in interview:

"Let me walk through the end-to-end system. It starts with query understanding — at the basic level that's spell correction and intent detection, but more advanced systems use LLMs to understand the customer's journey and dynamically route sourcing strategies.

Then multi-channel retrieval: I'd run BM25, semantic dual encoder, offline pre-computed tables, and graph-based retrieval in parallel. Each covers

different aspects — exact match, semantic similarity, proven historical patterns, and behavioral signals. Results are merged using Reciprocal Rank Fusion, which only uses ranks so you don't need to normalize scores across channels.

After fusion, a relevance model — a distilled cross-encoder — acts as a quality gate: anything below a relevance threshold is filtered regardless of revenue potential. Then the ranking model scores remaining candidates on multiple objectives: predicted CTR, predicted conversion rate, bid price, and relevance — probably via a deep network with feature crosses. Finally, an auction/re-ranking layer applies business constraints: budget pacing, frequency caps, diversity, and privacy filters.

The design principle throughout: fewer candidates at each stage → heavier model → higher quality per candidate. Total latency stays under 100ms."

Q15: Privacy-centric ads matching (Apple-specific)

Constraints:

- No cross-app tracking (IDFA opt-in rate ~25%)
- No user-level behavioral profiles shared across apps
- Differential privacy requirements

What exactly is restricted vs allowed:

ALLOWED (first-party, session-level, aggregated):

- Query text in current session — first-party, within App Store
- Apps browsed in current session (dwell time, clicks) — session-level, ephemeral
- App-to-app co-install graph (aggregated) — population-level, no individual ID
- Session-based intent refinement via LLM — on-session contextual, no persistence needed

NOT ALLOWED (cross-app, persistent, third-party):

- Cross-app behavior (what user did in Safari, Instagram) — needs IDFA
- Third-party DMP data (external audience profiles) — no third-party enrichment

LIMITED (needs special handling):

- Persistent user profile across sessions — requires differential privacy or on-device only
- Long-term user embedding (historical behavior) — on-device only, not server-side

What this limits for algorithm development:

Google/Meta can do → Apple cannot → Apple alternative:

- User embedding from full behavioral history → No persistent user profile → Session-

level embedding (ephemeral)

- Cross-app interest graph → Only App Store data → Within-store co-install graph
- Long-term personalized retrieval model → No user-level features server-side → Contextual retrieval + session adaptation
- Lookalike audience targeting → No individual-level matching → Cohort-level targeting with DP
- Retargeting (user saw X, show Y later) → No cross-session retargeting → Within-session journey only

How to do good matching under privacy:

1. Contextual signals only:

- Query text, search context, time of day, device type, OS version
- Current app category being browsed
- NO: browsing history, cross-app behavior, location history

2. On-device processing:

- Run lightweight models on-device
- Send only aggregated/anonymized signals to server
- Apple's on-device ML framework (Core ML) enables this

3. Differential privacy:

- Aggregate behavioral statistics with calibrated noise
- Know that "photo editor queries lead to 30% download rate for app X" without knowing WHO
- Apple's Private Relay / SKAdNetwork patterns

4. Cohort-based approaches:

- Group users into privacy-preserving cohorts by interest
- Similar to Google's Topics API but Apple's own implementation
- No individual-level targeting, only cohort-level

5. First-party contextual relevance:

- Heavy investment in query-app semantic matching (doesn't need user data)
- App quality signals (ratings, reviews, developer reputation)
- Category-level behavioral priors (not user-level)

LLM Session-based Orchestration — Fully Privacy-Compliant

Your idea of using LLM to understand customer journey within a session and adapt sourcing strategy is **completely compliant** with Apple's privacy model:

Step 1: User searches "photo editing"

- Standard multi-channel retrieval (no personalization yet)

- Shows: Lightroom, VSCO, Snapseed, PicsArt, Canva...

Step 2: User browses within session

- Clicks Lightroom page (dwell: 45s), goes back
- Clicks Photoshop page (dwell: 30s), goes back
- Skips VSCO, PicsArt (casual/filter apps)

Step 3: Session signal collected (first-party, ephemeral, no persistent ID)

- Interested in PRO-level tools (not casual filters)
- Specifically color grading / retouching

Step 4: LLM Journey Understanding (on-session, no stored profile)

- Input: original query + session clicks + dwell patterns
- Output: refined intent = "professional color grading software"
- Rewritten query: "photo color correction professional tool"
- Strategy: activate app-to-app retrieval (Lightroom as seed)
- Thresholds: higher quality bar, less diversity needed

Step 5: Re-retrieval with adapted strategy

- Semantic: encode "professional color grading" via ANN
- App-to-app: Lightroom co-install neighbors (Darkroom, Affinity Photo)
- Offline table: known pro photo editing apps

Step 6: Updated results shown

- Darkroom, Affinity Photo, Capture One, Pixelmator Pro

Why this is privacy-compliant:

- Only uses within-session, first-party signals (App Store is Apple's own platform)
- No IDFA needed, no cross-app tracking
- Session data can be ephemeral (discarded after session ends)
- LLM inference can run on-device if needed
- App-to-app graph is population-level aggregation (no individual data)

This is actually the OPTIMAL strategy under privacy constraints — since you can't build long-term user profiles, the best lever is real-time contextual understanding within the session.

This is actually a competitive advantage:

- Forces better contextual matching → better relevance
- Users trust the platform more → more engagement
- Advertisers get privacy-compliant targeting → regulatory safety

How to say it in interview:

"Apple's privacy constraints shape the system in specific ways. We can't do cross-app tracking, persistent user profiles, or third-party data enrichment. But what we CAN use — and this is the key insight — is first-party, session-level signals within the App Store itself.

So my approach would be: start with standard contextual retrieval on the first query. But as the user browses — clicks on Lightroom, spends time on Photoshop, skips the casual filter apps — I'd use an LLM to understand their evolving intent within that session. The LLM refines the query from generic 'photo editing' to specific 'professional color grading,' and dynamically adjusts the sourcing strategy — activating app-to-app retrieval based on what they've been browsing, tightening the relevance threshold.

This is fully privacy-compliant: it's all first-party, within-session, ephemeral data. No IDFA, no cross-app tracking, no persistent profile. The session ends, the data can be discarded. And it's actually the optimal strategy under these constraints — since we can't exploit long-term user history, real-time session understanding becomes our primary personalization lever.

For aggregate signals, I'd use differential privacy on population-level statistics — we can know that 'photo editor' queries lead to 30% download rate for Lightroom without knowing which user did what. And the co-install graph used for app-to-app retrieval is also population-level — 'users who installed Lightroom also installed Darkroom' requires no individual tracking."

Part 3: System Design Round Framework

Time allocation (45 min):

Phase	Time	Focus
Problem clarification & metrics	5 min	Scope, scale, latency requirements, success metrics
High-level architecture	5 min	Draw the multi-stage pipeline
Modeling deep dive	20 min	Feature engineering, model architecture, training, loss, evaluation
System & infrastructure	10 min	Serving, ANN index, latency, scaling, monitoring
Trade-offs & iteration	5 min	What to improve, A/B testing strategy

Modeling deep dive should cover:

- Feature engineering (what signals feed the model)
- Model architecture choice and WHY
- Training data collection and labeling
- Loss function design
- Offline evaluation and online A/B testing
- Failure modes and mitigations (cold start, position bias, relevance drift)

System/infra should cover:

- Real-time serving path vs offline computation
 - ANN index building and updating
 - Model update cadence
 - Monitoring and alerting (latency, relevance drift, CTR drops)
 - Scaling considerations (50K apps today → millions in future)
-

Key Takeaways for All Rounds

1. **Always tie back to Apple's constraints:** privacy, customer experience, premium brand
2. **Show end-to-end thinking:** not just the model, but data → training → serving → monitoring
3. **Quantify impact:** mention metrics (CTR + X%, Recall@500 = 95%, latency < 50ms)
4. **BM25 doesn't need training** — it's the unsupervised baseline that always works
5. **Dual encoder for retrieval, cross-encoder for ranking** — standard industry pattern
6. **Multi-objective optimization** — revenue vs relevance is THE core tension in ads
7. **Apple is earlier stage** — show you can build from scratch, not just optimize at scale